File Tree
해시 확인: certutil -hashfile <dist> SHA256
dist/
├── assets/
│   ├── index-av5gkzHq.css (11844 B / 017d5c46ea47)
│   └── index-CkN8R2zc.js (98313 B / 91c6a85e0695)
└── index.html (6983 B / a6648d94f2cc)

==== assets/index-av5gkzHq.css (11844 B / 017d5c46ea47) ====

```
*,
*:before,
*:after {
  box-sizing: border-box;
}
html {
  min-width: 1280px;
  color-scheme: light;
}
body {
  min-width: 1280px;
  min-height: 720px;
  margin: 0;
  font-family:
    Inter,
    ui-sans-serif,
    system-ui,
    -apple-system,
    BlinkMacSystemFont,
    Segoe UI,
    sans-serif;
  text-rendering: geometricPrecision;
}
button {
  font: inherit;
}
button:not(:disabled) {
  cursor: pointer;
}
button:disabled {
  cursor: not-allowed;
}
:root {
  --app-padding: 28px;
  --page: #f3eee3;
  --ink: #20252b;
  --muted: #66716f;
  --line: #c9c0b1;
  --panel: #fffaf0;
  --button: #263a3c;
  --button-text: #ffffff;
  --accent: #b94b36;
  --light-square: #eadbb7;
  --dark-square: #5f8a65;
  --last-move: #e0b84b;
  --legal: #264f8f;
  --capture: #b83232;
  --shadow: 0 18px 42px rgb(36 31 24 / 18%);
}
[hidden] {
  display: none !important;
}
.chess-app {
  --board-max-size: 560px;
  width: 100%;
  min-width: 1280px;
  height: 720px;
  min-height: 720px;
  display: grid;
  grid-template-columns: 230px var(--board-max-size) 118px 280px;
```

```
  gap: 10px;
  align-items: stretch;
  justify-content: center;
  overflow: hidden;
  padding: var(--app-padding);
  background: var(--page);
  color: var(--ink);
}
.board-stage {
  width: var(--board-max-size);
  align-self: center;
  justify-self: center;
}
.clock-panel {
  width: 100%;
  height: var(--board-max-size);
  min-width: 0;
  min-height: 0;
  align-self: center;
  justify-self: stretch;
  display: flex;
}
.chess-board {
  width: 100%;
  aspect-ratio: 1;
  display: grid;
  grid-template-columns: repeat(8, minmax(0, 1fr));
  overflow: hidden;
  border: 3px solid #2e3a32;
  border-radius: 8px;
  box-shadow: var(--shadow);
  background: #2e3a32;
}
.board-square {
  position: relative;
  width: 100%;
  aspect-ratio: 1;
  display: grid;
  place-items: center;
  min-width: 0;
  padding: 0;
  border: 0;
  color: var(--ink);
  transition:
    box-shadow 0.14s ease,
    filter 0.14s ease,
    transform 0.14s ease;
}
.board-square:not(:disabled):hover {
  filter: brightness(1.05);
}
.board-square.is-light {
  background: var(--light-square);
}
.board-square.is-dark {
  background: var(--dark-square);
}
.board-square.is-selected {
  box-shadow: inset 0 0 0 4px var(--legal);
}
.board-square.is-last-move {
  box-shadow: inset 0 0 0 4px #e0b84bc2;
}
.board-square.is-selected.is-last-move {
  box-shadow:
    inset 0 0 0 4px var(--legal),
    inset 0 0 0 8px #e0b84bad;
}
.board-square.is-legal:after,
.board-square.is-capture:after {
  position: absolute;
```

```
  z-index: 1;
  content: '';
  border-radius: 999px;
  pointer-events: none;
}
.board-square.is-legal:after {
  width: 28%;
  height: 28%;
  background: #264f8f6b;
}
.board-square.is-capture:after {
  inset: 12%;
  border: 3px solid rgba(184, 50, 50, 0.74);
}
.board-square.is-check {
  box-shadow: inset 0 0 0 5px var(--capture);
}
.chess-board.is-locked .board-square {
  filter: saturate(0.86);
}
.piece {
  z-index: 2;
  display: block;
  font-size: 4rem;
  line-height: 1;
  text-shadow: 0 2px 6px rgba(0, 0, 0, 0.2);
  -webkit-user-select: none;
  user-select: none;
}
.coordinate {
  position: absolute;
  z-index: 3;
  font-size: 0.7rem;
  font-weight: 800;
  line-height: 1;
  opacity: 0.72;
  pointer-events: none;
}
.coordinate-rank {
  top: 6px;
  left: 7px;
}
.coordinate-file {
  right: 7px;
  bottom: 6px;
}
.control-panel,
.info-panel {
  min-width: 0;
  min-height: 0;
  max-height: 664px;
  display: flex;
  flex-direction: column;
  gap: 14px;
  overflow: auto;
  overscroll-behavior: contain;
  scrollbar-gutter: stable;
}
.control-panel {
  justify-self: end;
}
.info-panel {
  justify-self: start;
}
.control-label {
  margin: 0;
  color: var(--muted);
  font-size: 0.78rem;
  font-weight: 800;
  letter-spacing: 0;
  text-transform: uppercase;
```

```
}
.control-group,
.status-panel,
.ai-candidate-panel,
.history-panel {
  display: grid;
  gap: 10px;
  padding-top: 16px;
  border-top: 1px solid var(--line);
}
.history-panel {
  min-height: 0;
  flex: 1;
  grid-template-rows: auto minmax(0, 1fr);
}
.segmented-control {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 4px;
  padding: 4px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fffaf0ad;
}
.segmented-control-three {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
.clock-preset-control {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 4px;
  padding: 4px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fffaf0ad;
}
.segmented-button,
.primary-button,
.ghost-button,
.promotion-button {
  min-height: 42px;
  border-radius: 6px;
  font-weight: 800;
  letter-spacing: 0;
}
.segmented-button {
  border: 0;
  background: transparent;
  color: var(--muted);
}
.segmented-button.is-active {
  background: var(--button);
  color: var(--button-text);
}
.clock-settings .segmented-button {
  min-height: 36px;
  padding-inline: 4px;
  font-size: 0.82rem;
}
.ai-settings {
  gap: 12px;
}
.range-control,
.toggle-control {
  display: grid;
  gap: 8px;
  min-width: 0;
  color: var(--ink);
  font-size: 0.92rem;
  font-weight: 800;
}
```

```
.range-control span,
.toggle-control {
  align-items: center;
}
.range-control span {
  display: flex;
  justify-content: space-between;
  gap: 10px;
}
.range-control strong {
  color: var(--accent);
  white-space: nowrap;
}
.range-control input[type='range'] {
  width: 100%;
  accent-color: var(--button);
}
.toggle-control {
  grid-template-columns: auto minmax(0, 1fr);
  padding: 10px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fffaf0bd;
}
.toggle-control input {
  width: 18px;
  height: 18px;
  accent-color: var(--button);
}
.action-row {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
}
.primary-button,
.ghost-button {
  border: 1px solid var(--button);
}
.primary-button {
  background: var(--button);
  color: var(--button-text);
}
.ghost-button {
  background: transparent;
  color: var(--button);
}
.primary-button:disabled,
.ghost-button:disabled,
.segmented-button:disabled {
  opacity: 0.46;
}
.game-status {
  min-height: 1.4em;
  color: var(--accent);
  font-size: 1.35rem;
  line-height: 1.2;
}
.clock-display {
  width: 100%;
  height: 100%;
  min-height: 0;
  min-width: 0;
  display: flex;
  flex-direction: column;
  gap: 10px;
  justify-content: space-between;
}
.clock-card {
  --clock-bg: rgb(255 250 240 / 74%);
  --clock-border: var(--line);
  --clock-side: var(--muted);
```

```
  --clock-time: var(--ink);
  width: 100%;
  min-block-size: 82px;
  max-block-size: 110px;
  min-width: 0;
  display: grid;
  place-content: center;
  gap: 4px;
  padding: 12px 8px;
  border: 1px solid var(--clock-border);
  border-radius: 8px;
  background: var(--clock-bg);
  text-align: center;
}
#white-clock {
  --clock-bg: #fffaf0;
  --clock-border: #c9c0b1;
  --clock-side: #66716f;
  --clock-time: #20252b;
}
#black-clock {
  --clock-bg: #263a3c;
  --clock-border: #172426;
  --clock-side: rgb(255 250 240 / 74%);
  --clock-time: #fffaf0;
}
.clock-card.is-active {
  border-color: var(--button);
  box-shadow:
    inset 0 0 0 2px #ffffff4d,
    0 0 0 2px #263a3c2e;
}
.clock-card.is-warning .clock-time {
  color: #d18b00;
}
.clock-card.is-danger .clock-time,
.clock-card.is-timeout .clock-time {
  color: var(--capture);
}
.clock-side {
  color: var(--clock-side);
  font-size: 0.72rem;
  font-weight: 800;
  line-height: 1;
}
.clock-time {
  font-variant-numeric: tabular-nums;
  color: var(--clock-time);
  font-size: 1.35rem;
  line-height: 1;
}
.status-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 8px;
  margin: 0;
}
.status-grid div {
  min-width: 0;
  padding: 10px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fffaf0bd;
}
.status-grid dt,
.status-grid dd {
  margin: 0;
}
.status-grid dt {
  color: var(--muted);
  font-size: 0.72rem;
```

```
  font-weight: 800;
}
.status-grid dd {
  margin-top: 4px;
  overflow-wrap: anywhere;
  font-size: 0.92rem;
  font-weight: 800;
}
.history-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}
.ai-candidate-panel {
  min-height: 0;
}
.ai-info-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 6px;
  margin: 0;
}
.ai-info-grid div {
  min-width: 0;
  padding: 8px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fffaf0bd;
}
.ai-info-grid dt,
.ai-info-grid dd {
  margin: 0;
}
.ai-info-grid dt {
  color: var(--muted);
  font-size: 0.64rem;
  font-weight: 800;
  text-transform: uppercase;
}
.ai-info-grid dd {
  margin-top: 3px;
  overflow: hidden;
  font-size: 0.8rem;
  font-weight: 800;
  line-height: 1.1;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.ai-info-grid dd[id='ai-info-nodes'],
.ai-info-grid dd[id='ai-info-score'],
.ai-info-grid dd[id='ai-info-depth'] {
  font-variant-numeric: tabular-nums;
}
.ai-candidate-header {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 8px;
}
.ai-candidate-meta {
  min-width: 0;
  overflow: hidden;
  color: var(--muted);
  font-size: 0.68rem;
  font-weight: 800;
  text-align: right;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.ai-candidate-table {
  min-height: 0;
```

```
  overflow: hidden;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fffaf0bd;
}
.ai-candidate-list {
  max-height: 172px;
  overflow: auto;
}
.ai-candidate-row {
  display: grid;
  grid-template-columns: minmax(3rem, 1.3fr) 2.6rem 3.2rem 3.2rem;
  gap: 6px;
  align-items: center;
  min-width: 0;
  padding: 6px 8px;
  border-bottom: 1px solid rgba(201, 192, 177, 0.58);
  font-size: 0.72rem;
  font-weight: 800;
  line-height: 1.15;
}
.ai-candidate-row:last-child {
  border-bottom: 0;
}
.ai-candidate-head {
  color: var(--muted);
  background: #fffaf0e0;
  font-size: 0.66rem;
  text-transform: uppercase;
}
.ai-candidate-row.is-selected {
  background: #b94b361f;
}
.ai-candidate-row.is-principal:not(.is-selected) {
  background: #263a3c14;
}
.ai-candidate-san,
.ai-candidate-score,
.ai-candidate-weight,
.ai-candidate-probability {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.ai-candidate-score,
.ai-candidate-weight,
.ai-candidate-probability {
  text-align: right;
  font-variant-numeric: tabular-nums;
}
.ai-candidate-badge {
  display: inline-block;
  margin-left: 4px;
  padding: 1px 3px;
  border-radius: 4px;
  background: var(--button);
  color: var(--button-text);
  font-size: 0.56rem;
  line-height: 1;
  vertical-align: 1px;
}
.move-list {
  min-height: 0;
  max-height: none;
  overflow: auto;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #fffaf0bd;
}
.move-row {
```

```
  display: grid;
  grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
  gap: 8px;
  padding: 8px 10px;
  border-bottom: 1px solid rgba(201, 192, 177, 0.62);
  font-weight: 800;
}
.move-row:last-child {
  border-bottom: 0;
}
.move-number {
  color: var(--muted);
}
.move-san {
  min-width: 0;
  overflow-wrap: anywhere;
}
.move-empty {
  margin: 0;
  padding: 14px 12px;
  color: var(--muted);
  font-weight: 700;
}
.promotion-overlay {
  position: fixed;
  inset: 0;
  z-index: 10;
  display: grid;
  place-items: center;
  padding: 20px;
  background: #15191c94;
}
.promotion-overlay[hidden] {
  display: none;
}
.promotion-dialog {
  width: 360px;
  display: grid;
  gap: 16px;
  padding: 20px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel);
  box-shadow: var(--shadow);
}
.promotion-dialog h2 {
  margin: 0;
  font-size: 1.1rem;
}
.promotion-choices {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 8px;
}
.promotion-button {
  min-height: 70px;
  border: 1px solid var(--button);
  background: #fff;
  color: var(--ink);
  font-size: 2.4rem;
  line-height: 1;
}

==== assets/index-CkN8R2zc.js (98313 B / 91c6a85e0695) ====
const MASK64 = 0xffffffffffffffffn;
function rotl(x, k) {
  return 0xffffffffffffffffn & ((x << k) | (x >> (64n - k)));
}
function wrappingMul(x, y) {
  return (x * y) & MASK64;
```

```
}
const rand = (function xoroshiro128(state) {
  return function () {
    let s0 = BigInt(state & MASK64),
      s1 = BigInt((state >> 64n) & MASK64);
    const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
    return (
      (s1 ^= s0),
      (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
      (s1 = rotl(s1, 37n)),
      (state = (s1 << 64n) | s0),
      result
    );
  };
})(0xa187eb39cdcaed8f31c4b365b102e01en),
PIECE_KEYS = Array.from({ length: 2 }, () =>
  Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand())),
),
EP_KEYS = Array.from({ length: 8 }, () => rand()),
CASTLING_KEYS = Array.from({ length: 16 }, () => rand()),
SIDE_KEY = rand(),
DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1';
class Move {
  color;
  from;
  to;
  piece;
  captured;
  promotion;
  flags;
  san;
  lan;
  before;
  after;
  constructor(chess, internal) {
    const {
      color: color,
      piece: piece,
      from: from,
      to: to,
      flags: flags,
      captured: captured,
      promotion: promotion,
    } = internal,
      fromAlgebraic = algebraic(from),
      toAlgebraic = algebraic(to);
    ((this.color = color),
      (this.piece = piece),
      (this.from = fromAlgebraic),
      (this.to = toAlgebraic),
      (this.san = chess._moveToSan(internal, chess._moves({ legal: !0 }))),
      (this.lan = fromAlgebraic + toAlgebraic),
      (this.before = chess.fen()),
      chess._makeMove(internal),
      (this.after = chess.fen()),
      chess._undoMove(),
      (this.flags = ''));
    for (const flag in BITS) BITS[flag] & flags && (this.flags += FLAGS[flag]);
    (captured && (this.captured = captured),
      promotion && ((this.promotion = promotion), (this.lan += promotion)));
  }
  isCapture() {
    return this.flags.indexOf(FLAGS.CAPTURE) > -1;
  }
  isPromotion() {
    return this.flags.indexOf(FLAGS.PROMOTION) > -1;
  }
  isEnPassant() {
    return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
  }
  isKingsideCastle() {
```

```
    return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
  }
  isQueensideCastle() {
    return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
  }
  isBigPawn() {
    return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
  }
}
const FLAGS = {
  NORMAL: 'n',
  CAPTURE: 'c',
  BIG_PAWN: 'b',
  EP_CAPTURE: 'e',
  PROMOTION: 'p',
  KSIDE_CASTLE: 'k',
  QSIDE_CASTLE: 'q',
  NULL_MOVE: '-',
},
BITS = {
  NORMAL: 1,
  CAPTURE: 2,
  BIG_PAWN: 4,
  EP_CAPTURE: 8,
  PROMOTION: 16,
  KSIDE_CASTLE: 32,
  QSIDE_CASTLE: 64,
  NULL_MOVE: 128,
},
Ox88 = {
  a8: 0,
  b8: 1,
  c8: 2,
  d8: 3,
  e8: 4,
  f8: 5,
  g8: 6,
  h8: 7,
  a7: 16,
  b7: 17,
  c7: 18,
  d7: 19,
  e7: 20,
  f7: 21,
  g7: 22,
  h7: 23,
  a6: 32,
  b6: 33,
  c6: 34,
  d6: 35,
  e6: 36,
  f6: 37,
  g6: 38,
  h6: 39,
  a5: 48,
  b5: 49,
  c5: 50,
  d5: 51,
  e5: 52,
  f5: 53,
  g5: 54,
  h5: 55,
  a4: 64,
  b4: 65,
  c4: 66,
  d4: 67,
  e4: 68,
  f4: 69,
  g4: 70,
  h4: 71,
  a3: 80,
```

```
    b3: 81,
    c3: 82,
    d3: 83,
    e3: 84,
    f3: 85,
    g3: 86,
    h3: 87,
    a2: 96,
    b2: 97,
    c2: 98,
    d2: 99,
    e2: 100,
    f2: 101,
    g2: 102,
    h2: 103,
    a1: 112,
    b1: 113,
    c1: 114,
    d1: 115,
    e1: 116,
    f1: 117,
    g1: 118,
    h1: 119,
  },
  PAWN_OFFSETS = { b: [16, 32, 17, 15], w: [-16, -32, -17, -15] },
  PIECE_OFFSETS = {
    n: [-18, -33, -31, -14, 18, 33, 31, 14],
    b: [-17, -15, 17, 15],
    r: [-16, 1, 16, -1],
    q: [-17, -16, -15, 1, 17, 16, 15, -1],
    k: [-17, -16, -15, 1, 17, 16, 15, -1],
  },
  ATTACKS = [
    20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20,
    0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20,
    0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 24, 24, 24, 24, 24, 24, 56,
    0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0,
    0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0,
    0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0,
    20,
  ],
  RAYS = [
    17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15,
    0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0, 0, 15,
    0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 16, 0, 15,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, -1,
    -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17, 0, 0, 0, 0, 0, 0,
    0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17,
    0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, -17, 0, 0, -15, 0, 0, 0, 0, 0, 0, -16, 0, 0,
    0, 0, 0, 0, -17,
  ],
  PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 },
  PROMOTIONS = ['n', 'b', 'r', 'q'],
  ROOKS = {
    w: [
      { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
      { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
    ],
    b: [
      { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
      { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
    ],
  },
  SECOND_RANK = { b: 1, w: 6 };
function rank(square) {
  return square >> 4;
}
function file(square) {
```

```
    return 15 & square;
}
function isDigit(c) {
    return -1 !== '0123456789'.indexOf(c);
}
function algebraic(square) {
    const f = file(square),
        r = rank(square);
    return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
}
function swapColor(color) {
    return 'w' === color ? 'b' : 'w';
}
function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
    const r = rank(to);
    if ('p' !== piece || (7 !== r && 0 !== r))
        moves.push({
            color: color,
            from: from,
            to: to,
            piece: piece,
            captured: captured,
            flags: flags,
        });
    else
        for (let i = 0; i < PROMOTIONS.length; i++) {
            const promotion = PROMOTIONS[i];
            moves.push({
                color: color,
                from: from,
                to: to,
                piece: piece,
                captured: captured,
                promotion: promotion,
                flags: flags | BITS.PROMOTION,
            });
        }
}
class Chess {
    _board = new Array(128);
    _turn = 'w';
    _kings = { w: -1, b: -1 };
    _epSquare = -1;
    _halfMoves = 0;
    _moveNumber = 0;
    _history = [];
    _castling = { w: 0, b: 0 };
    _hash = 0n;
    _positionCount = new Map();
    constructor(fen = DEFAULT_POSITION, { skipValidation: skipValidation = !1 } = {}) {
        this.load(fen, { skipValidation: skipValidation });
    }
    clear() {
        ((this._board = new Array(128)),
            (this._kings = { w: -1, b: -1 }),
            (this._turn = 'w'),
            (this._castling = { w: 0, b: 0 }),
            (this._epSquare = -1),
            (this._halfMoves = 0),
            (this._moveNumber = 1),
            (this._history = []),
            (this._hash = this._computeHash()),
            (this._positionCount = new Map()));
    }
    load(fen, { skipValidation: skipValidation = !1 } = {}) {
        let tokens = fen.split(/\s+/);
        if (tokens.length >= 2 && tokens.length < 6) {
            const adjustments = ['-', '-', '0', '1'];
            fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
        }
        if (((tokens = fen.split(/\s+/)), !skipValidation)) {
```

```
    const {ok: ok, error: error} = (function validateFen(fen) {
      const tokens = fen.split(/\s+/);
      if (6 !== tokens.length)
        return {ok: !1, error: 'Invalid FEN: must contain six space-delimited fields'};
      const moveNumber = parseInt(tokens[5], 10);
      if (isNaN(moveNumber) || moveNumber <= 0)
        return {ok: !1, error: 'Invalid FEN: move number must be a positive integer'};
      const halfMoves = parseInt(tokens[4], 10);
      if (isNaN(halfMoves) || halfMoves < 0)
        return {
          ok: !1,
          error: 'Invalid FEN: half move counter number must be a non-negative integer',
        };
      if (!/^(-|[abcdefgh][36])$/.test(tokens[3]))
        return {ok: !1, error: 'Invalid FEN: en-passant square is invalid'};
      if (/[^kKqQ-]/.test(tokens[2]))
        return {ok: !1, error: 'Invalid FEN: castling availability is invalid'};
      if (!/^(w|b)$/.test(tokens[1]))
        return {ok: !1, error: 'Invalid FEN: side-to-move is invalid'};
      const rows = tokens[0].split('/');
      if (8 !== rows.length)
        return {ok: !1, error: "Invalid FEN: piece data does not contain 8 '/'-delimited rows"};
      for (let i = 0; i < rows.length; i++) {
        let sumFields = 0,
          previousWasNumber = !1;
        for (let k = 0; k < rows[i].length; k++)
          if (isDigit(rows[i][k])) {
            if (previousWasNumber)
              return {ok: !1, error: 'Invalid FEN: piece data is invalid (consecutive number)'};
            ((sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0));
          } else {
            if (!/^[prnbqkPRNBQK]$/.test(rows[i][k]))
              return {ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)'};
            ((sumFields += 1), (previousWasNumber = !1));
          }
        if (8 !== sumFields)
          return {
            ok: !1,
            error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
          };
      }
      if (('3' === tokens[3][1] && 'w' === tokens[1]) || ('6' === tokens[3][1] && 'b' === tokens[1]))
        return {ok: !1, error: 'Invalid FEN: illegal en-passant square'};
      const kings = [
        {color: 'white', regex: /K/g},
        {color: 'black', regex: /k/g},
      ];
      for (const {color: color, regex: regex} of kings) {
        if (!regex.test(tokens[0]))
          return {ok: !1, error: `Invalid FEN: missing ${color} king`};
        if ((tokens[0].match(regex) || []).length > 1)
          return {ok: !1, error: `Invalid FEN: too many ${color} kings`};
      }
      return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
        ? {ok: !1, error: 'Invalid FEN: some pawns are on the edge rows'}
        : {ok: !0};
    })(fen);
    if (!ok) throw new Error(error);
  }
  const position = tokens[0];
  let square = 0;
  this.clear();
  for (let i = 0; i < position.length; i++) {
    const piece = position.charAt(i);
    if ('/' === piece) square += 8;
    else if (isDigit(piece)) square += parseInt(piece, 10);
    else {
      const color = piece < 'a' ? 'w' : 'b';
      (this._put({type: piece.toLowerCase(), color: color}, algebraic(square)), square++);
    }
  }
}
```

```
    ((this._turn = tokens[1]),
    tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
    tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),
    tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
    tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
    (this._epSquare = '-' === tokens[3] ? -1 : Ox88[tokens[3]]),
    (this._halfMoves = parseInt(tokens[4], 10)),
    (this._moveNumber = parseInt(tokens[5], 10)),
    (this._hash = this._computeHash()),
    this._incPositionCount());
  }
  fen({ forceEnpassantSquare: forceEnpassantSquare = !1 } = {}) {
    let empty = 0,
      fen = '';
    for (let i = Ox88.a8; i <= Ox88.h1; i++) {
      if (this._board[i]) {
        empty > 0 && ((fen += empty), (empty = 0));
        const { color: color, type: piece } = this._board[i];
        fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
      } else empty++;
      (i + 1) & 136 &&
        (empty > 0 && (fen += empty), i !== Ox88.h1 && (fen += '/'), (empty = 0), (i += 8));
    }
    let castling = '';
    (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
      this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
      this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
      this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
      (castling = castling || '-'));
    let epSquare = '-';
    if (-1 !== this._epSquare)
      if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
      else {
        const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
          squares = [bigPawnSquare + 1, bigPawnSquare - 1];
        for (const square of squares) {
          if (136 & square) continue;
          const color = this._turn;
          if (this._board[square]?.color === color && 'p' === this._board[square]?.type) {
            this._makeMove({
              color: color,
              from: square,
              to: this._epSquare,
              piece: 'p',
              captured: 'p',
              flags: BITS.EP_CAPTURE,
            });
            const isLegal = !this._isKingAttacked(color);
            if ((this._undoMove(), isLegal)) {
              epSquare = algebraic(this._epSquare);
              break;
            }
          }
        }
      }
    return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
  }
  _pieceKey(i) {
    if (!this._board[i]) return 0n;
    const { color: color, type: type } = this._board[i],
      typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
    return PIECE_KEYS[{ w: 0, b: 1 }[color]][typeIndex][i];
  }
  _epKey() {
    return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
  }
  _castlingKey() {
    const index = (this._castling.w >> 5) | (this._castling.b >> 3);
    return CASTLING_KEYS[index];
  }
  _computeHash() {
```

```
	let hash = 0n;
	for (let i = 0x88.a8; i <= 0x88.h1; i++)
		136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));
	return (
		(hash ^= this._epKey()),
		(hash ^= this._castlingKey()),
		'b' === this._turn && (hash ^= SIDE_KEY),
		hash
	);
}
reset() {
	this.load(DEFAULT_POSITION);
}
get(square) {
	return this._board[0x88[square]];
}
_set(sq, piece) {
	((this._hash ^= this._pieceKey(sq)),
		(this._board[sq] = piece),
		(this._hash ^= this._pieceKey(sq)));
}
_put({ type: type, color: color }, square) {
	if (-1 === 'pnbrqkPNBRQK'.indexOf(type.toLowerCase())) return !1;
	if (!(square in 0x88)) return !1;
	const sq = 0x88[square];
	if ('k' === type && -1 !== this._kings[color] && this._kings[color] !== sq) return !1;
	const currentPieceOnSquare = this._board[sq];
	return (
		currentPieceOnSquare &&
			'k' === currentPieceOnSquare.type &&
			(this._kings[currentPieceOnSquare.color] = -1),
		this._set(sq, { type: type, color: color }),
		'k' === type && (this._kings[color] = sq),
		!0
	);
}
_clear(sq) {
	((this._hash ^= this._pieceKey(sq)), delete this._board[sq]);
}
_attacked(color, square, verbose) {
	const attackers = [];
	for (let i = 0x88.a8; i <= 0x88.h1; i++) {
		if (136 & i) {
			i += 7;
			continue;
		}
		if (void 0 === this._board[i] || this._board[i].color !== color) continue;
		const piece = this._board[i],
			difference = i - square;
		if (0 === difference) continue;
		const index = difference + 119;
		if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
			if ('p' === piece.type) {
				if ((difference > 0 && 'w' === piece.color) || (difference <= 0 && 'b' === piece.color)) {
					if (!verbose) return !0;
					attackers.push(algebraic(i));
				}
				continue;
			}
			if ('n' === piece.type || 'k' === piece.type) {
				if (verbose) {
					attackers.push(algebraic(i));
					continue;
				}
				return !0;
			}
			const offset = RAYS[index];
			let j = i + offset,
				blocked = !1;
			for (; j !== square; ) {
				if (null !== this._board[j]) {
```

```
            blocked = !0;
            break;
          }
          j += offset;
        }
        if (!blocked) {
          if (verbose) {
            attackers.push(algebraic(i));
            continue;
          }
          return !0;
        }
      }
    }
    return !!verbose && attackers;
  }
  attackers(square, attackedBy) {
    return attackedBy
      ? this._attacked(attackedBy, Ox88[square], !0)
      : this._attacked(this._turn, Ox88[square], !0);
  }
  _isKingAttacked(color) {
    const square = this._kings[color];
    return -1 !== square && this._attacked(swapColor(color), square);
  }
  hash() {
    return this._hash.toString(16);
  }
  isAttacked(square, attackedBy) {
    return this._attacked(attackedBy, Ox88[square]);
  }
  isCheck() {
    return this._isKingAttacked(this._turn);
  }
  inCheck() {
    return this.isCheck();
  }
  isCheckmate() {
    return this.isCheck() && 0 === this._moves().length;
  }
  isStalemate() {
    return !this.isCheck() && 0 === this._moves().length;
  }
  isInsufficientMaterial() {
    const pieces = { b: 0, n: 0, r: 0, q: 0, k: 0, p: 0 },
      bishops = [];
    let numPieces = 0,
      squareColor = 0;
    for (let i = 0x88.a8; i <= 0x88.h1; i++) {
      if (((squareColor = (squareColor + 1) % 2), 136 & i)) {
        i += 7;
        continue;
      }
      const piece = this._board[i];
      piece &&
        ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
        'b' === piece.type && bishops.push(squareColor),
        numPieces++);
    }
    if (2 === numPieces) return !0;
    if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
    if (numPieces === pieces.b + 2) {
      let sum = 0;
      const len = bishops.length;
      for (let i = 0; i < len; i++) sum += bishops[i];
      if (0 === sum || sum === len) return !0;
    }
    return !1;
  }
  isThreefoldRepetition() {
    return this._getPositionCount(this._hash) >= 3;
```

```
  }
  isDrawByFiftyMoves() {
    return this._halfMoves >= 100;
  }
  isDraw() {
    return (
      this.isDrawByFiftyMoves() ||
      this.isStalemate() ||
      this.isInsufficientMaterial() ||
      this.isThreefoldRepetition()
    );
  }
  isGameOver() {
    return this.isCheckmate() || this.isDraw();
  }
  moves({ verbose: verbose = !1, square: square, piece: piece } = {}) {
    const moves = this._moves({ square: square, piece: piece });
    return verbose
      ? moves.map((move) => new Move(this, move))
      : moves.map((move) => this._moveToSan(move, moves));
  }
  _moves({ legal: legal = !0, piece: piece, square: square } = {}) {
    const forSquare = square ? square.toLowerCase() : void 0,
      forPiece = piece?.toLowerCase(),
      moves = [],
      us = this._turn,
      them = swapColor(us);
    let firstSquare = 0x88.a8,
      lastSquare = 0x88.h1,
      singleSquare = !1;
    if (forSquare) {
      if (!(forSquare in 0x88)) return [];
      ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
    }
    for (let from = firstSquare; from <= lastSquare; from++) {
      if (136 & from) {
        from += 7;
        continue;
      }
      if (!this._board[from] || this._board[from].color === them) continue;
      const { type: type } = this._board[from];
      let to;
      if ('p' === type) {
        if (forPiece && forPiece !== type) continue;
        ((to = from + PAWN_OFFSETS[us][0]),
          this._board[to] ||
            (addMove(moves, us, from, to, 'p'),
            (to = from + PAWN_OFFSETS[us][1]),
            SECOND_RANK[us] !== rank(from) ||
              this._board[to] ||
              addMove(moves, us, from, to, 'p', void 0, BITS.BIG_PAWN)));
        for (let j = 2; j < 4; j++)
          ((to = from + PAWN_OFFSETS[us][j]),
            136 & to ||
              (this._board[to]?.color === them
                ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
                : to === this._epSquare &&
                  addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE)));
      } else {
        if (forPiece && forPiece !== type) continue;
        for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
          const offset = PIECE_OFFSETS[type][j];
          for (to = from; (to += offset), !(136 & to); ) {
            if (this._board[to]) {
              if (this._board[to].color === us) break;
              addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
              break;
            }
            if ((addMove(moves, us, from, to, type), 'n' === type || 'k' === type)) break;
          }
        }
      }
```

```
646 |  |  |  |  }
647 |  |  |  }
648 |  |  |  if (
649 |  |  |  |  !(
650 |  |  |  |  |  (void 0 !== forPiece && 'k' !== forPiece) ||
651 |  |  |  |  |  (singleSquare && lastSquare !== this._kings[us])
652 |  |  |  |  )
653 |  |  |  ) {
654 |  |  |  |  if (this._castling[us] & BITS.KSIDE_CASTLE) {
655 |  |  |  |  |  const castlingFrom = this._kings[us],
656 |  |  |  |  |  |  castlingTo = castlingFrom + 2;
657 |  |  |  |  |  this._board[castlingFrom + 1] ||
658 |  |  |  |  |  |  this._board[castlingTo] ||
659 |  |  |  |  |  |  this._attacked(them, this._kings[us]) ||
660 |  |  |  |  |  |  this._attacked(them, castlingFrom + 1) ||
661 |  |  |  |  |  |  this._attacked(them, castlingTo) ||
662 |  |  |  |  |  |  addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
663 |  |  |  |  }
664 |  |  |  |  if (this._castling[us] & BITS.QSIDE_CASTLE) {
665 |  |  |  |  |  const castlingFrom = this._kings[us],
666 |  |  |  |  |  |  castlingTo = castlingFrom - 2;
667 |  |  |  |  |  this._board[castlingFrom - 1] ||
668 |  |  |  |  |  |  this._board[castlingFrom - 2] ||
669 |  |  |  |  |  |  this._board[castlingFrom - 3] ||
670 |  |  |  |  |  |  this._attacked(them, this._kings[us]) ||
671 |  |  |  |  |  |  this._attacked(them, castlingFrom - 1) ||
672 |  |  |  |  |  |  this._attacked(them, castlingTo) ||
673 |  |  |  |  |  |  addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
674 |  |  |  |  }
675 |  |  |  }
676 |  |  |  if (!legal || -1 === this._kings[us]) return moves;
677 |  |  |  const legalMoves = [];
678 |  |  |  for (let i = 0, len = moves.length; i < len; i++)
679 |  |  |  |  (this._makeMove(moves[i]),
680 |  |  |  |  |  this._isKingAttacked(us) || legalMoves.push(moves[i]),
681 |  |  |  |  |  this._undoMove());
682 |  |  |  return legalMoves;
683 |  |  }
684 |  |  move(move) {
685 |  |  |  let moveObj = null;
686 |  |  |  const moves = this._moves();
687 |  |  |  for (let i = 0, len = moves.length; i < len; i++)
688 |  |  |  |  if (
689 |  |  |  |  |  move.from === algebraic(moves[i].from) &&
690 |  |  |  |  |  move.to === algebraic(moves[i].to) &&
691 |  |  |  |  |  (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
692 |  |  |  |  ) {
693 |  |  |  |  |  moveObj = moves[i];
694 |  |  |  |  |  break;
695 |  |  |  |  }
696 |  |  |  if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
697 |  |  |  const prettyMove = new Move(this, moveObj);
698 |  |  |  return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
699 |  |  }
700 |  |  _push(move) {
701 |  |  |  this._history.push({
702 |  |  |  |  move: move,
703 |  |  |  |  kings: { b: this._kings.b, w: this._kings.w },
704 |  |  |  |  turn: this._turn,
705 |  |  |  |  castling: { b: this._castling.b, w: this._castling.w },
706 |  |  |  |  epSquare: this._epSquare,
707 |  |  |  |  halfMoves: this._halfMoves,
708 |  |  |  |  moveNumber: this._moveNumber,
709 |  |  |  });
710 |  |  }
711 |  |  _movePiece(from, to) {
712 |  |  |  ((this._hash ^= this._pieceKey(from)),
713 |  |  |  |  (this._board[to] = this._board[from]),
714 |  |  |  |  delete this._board[from],
715 |  |  |  |  (this._hash ^= this._pieceKey(to)));
716 |  |  }
```

```
717 |  |  _makeMove(move) {
718 |  |  |  const us = this._turn,
719 |  |  |  |  them = swapColor(us);
720 |  |  |  if ((this._push(move), move.flags & BITS.NULL_MOVE))
721 |  |  |  |  return (
722 |  |  |  |  |  'b' === us && this._moveNumber++,
723 |  |  |  |  |  this._halfMoves++,
724 |  |  |  |  |  (this._turn = them),
725 |  |  |  |  |  void (this._epSquare = -1)
726 |  |  |  |  );
727 |  |  |  if (
728 |  |  |  |  ((this._hash ^= this._epKey()),
729 |  |  |  |  (this._hash ^= this._castlingKey()),
730 |  |  |  |  move.captured && (this._hash ^= this._pieceKey(move.to)),
731 |  |  |  |  this._movePiece(move.from, move.to),
732 |  |  |  |  move.flags & BITS.EP_CAPTURE &&
733 |  |  |  |  |  ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
734 |  |  |  |  move.promotion &&
735 |  |  |  |  |  (this._clear(move.to), this._set(move.to, { type: move.promotion, color: us })),
736 |  |  |  |  'k' === this._board[move.to].type)
737 |  |  |  ) {
738 |  |  |  |  if (((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE)) {
739 |  |  |  |  |  const castlingTo = move.to - 1,
740 |  |  |  |  |  |  castlingFrom = move.to + 1;
741 |  |  |  |  |  this._movePiece(castlingFrom, castlingTo);
742 |  |  |  |  } else if (move.flags & BITS.QSIDE_CASTLE) {
743 |  |  |  |  |  const castlingTo = move.to + 1,
744 |  |  |  |  |  |  castlingFrom = move.to - 2;
745 |  |  |  |  |  this._movePiece(castlingFrom, castlingTo);
746 |  |  |  |  }
747 |  |  |  |  this._castling[us] = 0;
748 |  |  |  }
749 |  |  |  if (this._castling[us])
750 |  |  |  |  for (let i = 0, len = ROOKS[us].length; i < len; i++)
751 |  |  |  |  |  if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
752 |  |  |  |  |  |  this._castling[us] ^= ROOKS[us][i].flag;
753 |  |  |  |  |  |  break;
754 |  |  |  |  |  }
755 |  |  |  if (this._castling[them])
756 |  |  |  |  for (let i = 0, len = ROOKS[them].length; i < len; i++)
757 |  |  |  |  |  if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
758 |  |  |  |  |  |  this._castling[them] ^= ROOKS[them][i].flag;
759 |  |  |  |  |  |  break;
760 |  |  |  |  |  }
761 |  |  |  if (((this._hash ^= this._castlingKey()), move.flags & BITS.BIG_PAWN)) {
762 |  |  |  |  let epSquare;
763 |  |  |  |  (epSquare = 'b' === us ? move.to - 16 : move.to + 16),
764 |  |  |  |  |  ((move.to - 1) & 136) !==
765 |  |  |  |  |  |  'p' !== this._board[move.to - 1]?.type ||
766 |  |  |  |  |  |  this._board[move.to - 1]?.color !== them) &&
767 |  |  |  |  |  ((move.to + 1) & 136) !==
768 |  |  |  |  |  |  'p' !== this._board[move.to + 1]?.type ||
769 |  |  |  |  |  |  this._board[move.to + 1]?.color !== them)
770 |  |  |  |  |  ? (this._epSquare = -1)
771 |  |  |  |  |  : ((this._epSquare = epSquare), (this._hash ^= this._epKey()));
772 |  |  |  } else this._epSquare = -1;
773 |  |  |  ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
774 |  |  |  |  ? (this._halfMoves = 0)
775 |  |  |  |  : this._halfMoves++,
776 |  |  |  |  'b' === us && this._moveNumber++,
777 |  |  |  |  (this._turn = them),
778 |  |  |  |  (this._hash ^= SIDE_KEY));
779 |  |  }
780 |  |  undo() {
781 |  |  |  const hash = this._hash,
782 |  |  |  |  move = this._undoMove();
783 |  |  |  if (move) {
784 |  |  |  |  const prettyMove = new Move(this, move);
785 |  |  |  |  return (this._decPositionCount(hash), prettyMove);
786 |  |  |  }
787 |  |  |  return null;
```

```
788 |  |  }
789 |  |  _undoMove() {
790 |  |  |  const old = this._history.pop();
791 |  |  |  if (void 0 === old) return null;
792 |  |  |  ((this._hash ^= this._epKey()), (this._hash ^= this._castlingKey()));
793 |  |  |  const move = old.move;
794 |  |  |  ((this._kings = old.kings),
795 |  |  |  |  (this._turn = old.turn),
796 |  |  |  |  (this._castling = old.castling),
797 |  |  |  |  (this._epSquare = old.epSquare),
798 |  |  |  |  (this._halfMoves = old.halfMoves),
799 |  |  |  |  (this._moveNumber = old.moveNumber),
800 |  |  |  |  (this._hash ^= this._epKey()),
801 |  |  |  |  (this._hash ^= this._castlingKey()),
802 |  |  |  |  (this._hash ^= SIDE_KEY));
803 |  |  |  const us = this._turn,
804 |  |  |  |  them = swapColor(us);
805 |  |  |  if (move.flags & BITS.NULL_MOVE) return move;
806 |  |  |  if (
807 |  |  |  |  (this._movePiece(move.to, move.from),
808 |  |  |  |  move.piece && (this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
809 |  |  |  |  move.captured)
810 |  |  |  )
811 |  |  |  |  if (move.flags & BITS.EP_CAPTURE) {
812 |  |  |  |  |  let index;
813 |  |  |  |  |  ((index = 'b' === us ? move.to - 16 : move.to + 16),
814 |  |  |  |  |  |  this._set(index, { type: 'p', color: them }));
815 |  |  |  |  } else this._set(move.to, { type: move.captured, color: them });
816 |  |  |  if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
817 |  |  |  |  let castlingTo, castlingFrom;
818 |  |  |  |  (move.flags & BITS.KSIDE_CASTLE
819 |  |  |  |  |  ? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
820 |  |  |  |  |  : ((castlingTo = move.to - 2), (castlingFrom = move.to + 1)),
821 |  |  |  |  |  this._movePiece(castlingFrom, castlingTo));
822 |  |  |  }
823 |  |  |  return move;
824 |  |  }
825 |  |  _moveToSan(move, moves) {
826 |  |  |  let output = '';
827 |  |  |  if (move.flags & BITS.KSIDE_CASTLE) output = '0-0';
828 |  |  |  else if (move.flags & BITS.QSIDE_CASTLE) output = '0-0-0';
829 |  |  |  else {
830 |  |  |  |  if ('p' !== move.piece) {
831 |  |  |  |  |  const disambiguator = (function getDisambiguator(move, moves) {
832 |  |  |  |  |  |  const from = move.from,
833 |  |  |  |  |  |  |  to = move.to,
834 |  |  |  |  |  |  |  piece = move.piece;
835 |  |  |  |  |  |  let ambiguities = 0,
836 |  |  |  |  |  |  |  sameRank = 0,
837 |  |  |  |  |  |  |  sameFile = 0;
838 |  |  |  |  |  |  for (let i = 0, len = moves.length; i < len; i++) {
839 |  |  |  |  |  |  |  const ambigFrom = moves[i].from,
840 |  |  |  |  |  |  |  |  ambigTo = moves[i].to;
841 |  |  |  |  |  |  |  piece === moves[i].piece &&
842 |  |  |  |  |  |  |  |  from !== ambigFrom &&
843 |  |  |  |  |  |  |  |  to === ambigTo &&
844 |  |  |  |  |  |  |  |  (ambiguities++,
845 |  |  |  |  |  |  |  |  rank(from) === rank(ambigFrom) && sameRank++,
846 |  |  |  |  |  |  |  |  file(from) === file(ambigFrom) && sameFile++);
847 |  |  |  |  |  |  }
848 |  |  |  |  |  |  return ambiguities > 0
849 |  |  |  |  |  |  |  ? sameRank > 0 && sameFile > 0
850 |  |  |  |  |  |  |  |  ? algebraic(from)
851 |  |  |  |  |  |  |  |  : sameFile > 0
852 |  |  |  |  |  |  |  |  |  ? algebraic(from).charAt(1)
853 |  |  |  |  |  |  |  |  |  : algebraic(from).charAt(0)
854 |  |  |  |  |  |  |  : '';
855 |  |  |  |  |  })(move, moves);
856 |  |  |  |  |  output += move.piece.toUpperCase() + disambiguator;
857 |  |  |  |  }
858 |  |  |  |  (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&
```

```
        ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
        (output += algebraic(move.to)),
        move.promotion && (output += '=' + move.promotion.toUpperCase()));
    }
    return (
      this._makeMove(move),
      this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
      this._undoMove(),
      output
    );
  }
  turn() {
    return this._turn;
  }
  board() {
    const output = [];
    let row = [];
    for (let i = 0x88.a8; i <= 0x88.h1; i++)
      (null == this._board[i]
        ? row.push(null)
        : row.push({
            square: algebraic(i),
            type: this._board[i].type,
            color: this._board[i].color,
          }),
      (i + 1) & 136 && (output.push(row), (row = []), (i += 8)));
    return output;
  }
  squareColor(square) {
    if (square in 0x88) {
      const sq = 0x88[square];
      return (rank(sq) + file(sq)) % 2 == 0 ? 'light' : 'dark';
    }
    return null;
  }
  history({ verbose: verbose = !1 } = {}) {
    const reversedHistory = [],
      moveHistory = [];
    for (; this._history.length > 0; ) reversedHistory.push(this._undoMove());
    for (;;) {
      const move = reversedHistory.pop();
      if (!move) break;
      (verbose
        ? moveHistory.push(new Move(this, move))
        : moveHistory.push(this._moveToSan(move, this._moves())),
        this._makeMove(move));
    }
    return moveHistory;
  }
  _getPositionCount(hash) {
    return this._positionCount.get(hash) ?? 0;
  }
  _incPositionCount() {
    this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
  }
  _decPositionCount(hash) {
    const currentCount = this._positionCount.get(hash) ?? 0;
    1 === currentCount
      ? this._positionCount.delete(hash)
      : this._positionCount.set(hash, currentCount - 1);
  }
  moveNumber() {
    return this._moveNumber;
  }
}
const AI_PRESETS = {
  easy: { preset: 'easy', maxDepth: 2, timeLimitMs: 750, randomness: 30, quiescence: !0 },
  normal: { preset: 'normal', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
  hard: { preset: 'hard', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
},
DEFAULT_AI_SETTINGS = { ...AI_PRESETS.normal };
```

```
function normalizeAiSettings(settings) {
  const timeLimitMs = clamp(Math.round(settings.timeLimitMs), 500, 1e4),
    softTimeLimitMs =
      void 0 === settings.softTimeLimitMs
        ? void 0
        : Math.min(timeLimitMs, clamp(Math.round(settings.softTimeLimitMs), 500, 1e4));
  return {
    preset: settings.preset,
    maxDepth: ((depth = settings.maxDepth), clamp(Math.round(depth), 1, 5)),
    timeLimitMs: timeLimitMs,
    ...(void 0 === softTimeLimitMs ? {} : { softTimeLimitMs: softTimeLimitMs }),
    randomness: 30,
    quiescence: !0,
  };
  var depth;
}
function clamp(value, min, max) {
  return Math.min(max, Math.max(min, value));
}
const jsContent =
  '!function(){"use strict";const MASK64=0xffffffffffffffffn;function rotl(x,k){return 0xffffffffffffffffn&(x<<k|x>>64n-k
  )}function wrappingMul(x,y){return x*y&MASK64}const rand=function xoroshiro128(state){return function(){let s0=BigInt(sta
  te&MASK64),s1=BigInt(state>>64n&MASK64);const result=wrappingMul(rotl(wrappingMul(s0,5n),7n),9n);return s1^=s0,s0=(rotl(s
  0,24n)^s1^s1<<16n)&MASK64,s1=rotl(s1,37n),state=s1<<64n|s0,result}}(0xa187eb39cdcaed8f31c4b365b102e01en),PIECE_KEYS=Array
  .from({length:2},()=>Array.from({length:6},()=>Array.from({length:128},()=>rand()))),EP_KEYS=Array.from({length:8},()=>ra
  nd()),CASTLING_KEYS=Array.from({length:16},()=>rand()),SIDE_KEY=rand(),DEFAULT_POSITION="rnbqkbnr/pppppppp/8/8/8/8/PPPPPP
  PP/RNBQKBNR w KQkq - 0 1";class Move{color;from;to;piece;captured;promotion;flags;san;lan;before;after;constructor(chess,
  internal){const{color:color,piece:piece,from:from,to:to,flags:flags,captured:captured,promotion:promotion}=internal,fromA
  lgebraic=algebraic(from),toAlgebraic=algebraic(to);this.color=color,this.piece=piece,this.from=fromAlgebraic,this.to=toAl
  gebraic,this.san=chess._moveToSan(internal,chess._moves({legal:!0})),this.lan=fromAlgebraic+toAlgebraic,this.before=chess
  .fen(),chess._makeMove(internal),this.after=chess.fen(),chess._undoMove(),this.flags="";for(const flag in BITS)BITS[flag]
  &flags&&(this.flags+=FLAGS[flag]);captured&&(this.captured=captured),promotion&&(this.promotion=promotion,this.lan+=promo
  tion)}isCapture(){return this.flags.indexOf(FLAGS.CAPTURE)>-1}isPromotion(){return this.flags.indexOf(FLAGS.PROMOTION)>-1
  }isEnPassant(){return this.flags.indexOf(FLAGS.EP_CAPTURE)>-1}isKingsideCastle(){return this.flags.indexOf(FLAGS.KSIDE_CA
  STLE)>-1}isQueensideCastle(){return this.flags.indexOf(FLAGS.QSIDE_CASTLE)>-1}isBigPawn(){return this.flags.indexOf(FLAGS
  .BIG_PAWN)>-1}}const FLAGS={NORMAL:"n",CAPTURE:"c",BIG_PAWN:"b",EP_CAPTURE:"e",PROMOTION:"p",KSIDE_CASTLE:"k",QSIDE_CASTL
  E:"q",NULL_MOVE:"-"},BITS={NORMAL:1,CAPTURE:2,BIG_PAWN:4,EP_CAPTURE:8,PROMOTION:16,KSIDE_CASTLE:32,QSIDE_CASTLE:64,NULL_M
  OVE:128},Ox88={a8:0,b8:1,c8:2,d8:3,e8:4,f8:5,g8:6,h8:7,a7:16,b7:17,c7:18,d7:19,e7:20,f7:21,g7:22,h7:23,a6:32,b6:33,c6:34,
  d6:35,e6:36,f6:37,g6:38,h6:39,a5:48,b5:49,c5:50,d5:51,e5:52,f5:53,g5:54,h5:55,a4:64,b4:65,c4:66,d4:67,e4:68,f4:69,g4:70,h
  4:71,a3:80,b3:81,c3:82,d3:83,e3:84,f3:85,g3:86,h3:87,a2:96,b2:97,c2:98,d2:99,e2:100,f2:101,g2:102,h2:103,a1:112,b1:113,c1
  :114,d1:115,e1:116,f1:117,g1:118,h1:119},PAWN_OFFSETS={b:[16,32,17,15],w:[-16,-32,-17,-15]},PIECE_OFFSETS={n:[-18,-33,-31
  ,-14,18,33,31,14],b:[-17,-15,17,15],r:[-16,1,16,-1],q:[-17,-16,-15,1,17,16,15,-1],k:[-17,-16,-15,1,17,16,15,-1]},ATTACKS=
  [20,0,0,0,0,0,0,24,0,0,0,0,0,0,20,0,0,20,0,0,0,0,0,24,0,0,0,0,0,20,0,0,0,0,20,0,0,0,0,24,0,0,0,0,20,0,0,0,0,0,20,0,0,0,
  24,0,0,0,20,0,0,0,0,0,0,0,20,0,0,24,0,0,20,0,0,0,0,0,0,0,0,0,20,2,24,2,20,0,0,0,0,0,0,0,0,0,0,2,53,56,53,2,0,0,0,0,
  0,0,24,24,24,24,24,24,56,0,56,24,24,24,24,24,24,0,0,0,0,0,0,2,53,56,53,2,0,0,0,0,0,0,0,0,0,0,20,2,24,2,20,0,0,0,0,0,0,0
  ,0,0,0,20,0,0,24,0,0,20,0,0,0,0,0,0,0,20,0,0,0,24,0,0,0,20,0,0,0,0,0,0,20,0,0,0,0,24,0,0,0,0,20,0,0,0,0,20,0,0,0,0,0,24
  ,0,0,0,0,0,20,0,0,20,0,0,0,0,0,0,24,0,0,0,0,0,0,20],RAYS=[17,0,0,0,0,0,0,16,0,0,0,0,0,0,15,0,0,17,0,0,0,0,0,16,0,0,0,0,0,
  15,0,0,0,0,17,0,0,0,0,16,0,0,0,0,15,0,0,0,0,0,0,17,0,0,0,16,0,0,0,15,0,0,0,0,0,0,0,0,0,0,0
  ,17,0,16,0,15,0,0,0,0,0,0,0,0,0,0,0,17,16,15,0,0,0,0,0,0,0,1,1,1,1,1,1,1,0,-1,-1,-1,-1,-1,-1,-1,0,0,0,0,0,0,0,-15,-16,-
  17,0,0,0,0,0,0,0,0,0,0,0,0,-15,0,-16,0,-17,0,0,0,0,0,0,0,0,0,0,-15,0,0,-16,0,0,-17,0,0,0,0,0,0,0,0,-15,0,0,0,-16,0,0,0,-1
  7,0,0,0,0,0,0,-15,0,0,0,0,0,-16,0,0,0,0,0,-17,0,0,0,0,0,-15,0,0,0,0,0,0,-16,0,0,0,0,0,0,-17,0,0,0,-15,0,0,0,0,0,0,-16,0,0,0,0,0,0,-17
  ],PIECE_MASKS={p:1,n:2,b:4,r:8,q:16,k:32},PROMOTIONS=["n","b","r","q"],ROOKS={w:[{square:Ox88.a1,flag:BITS.QSIDE_CASTLE},
  {square:Ox88.h1,flag:BITS.KSIDE_CASTLE}],b:[{square:Ox88.a8,flag:BITS.QSIDE_CASTLE},{square:Ox88.h8,flag:BITS.KSIDE_CASTL
  E}]},SECOND_RANK={b:1,w:6};function rank(square){return square>>4}function file(square){return 15&square}function isDigit
  (c){return-1!="0123456789".indexOf(c)}function algebraic(square){const f=file(square),r=rank(square);return"abcdefgh".su
  bstring(f,f+1)+"87654321".substring(r,r+1)}function swapColor(color){return"w"===color?"b":"w"}function addMove(moves,col
  or,from,to,piece,captured=void 0,flags=BITS.NORMAL){const r=rank(to);if("p"!=piece||7!=r&&0!=r)moves.push({color:color
  ,from:from,to:to,piece:piece,captured:captured,flags:flags});else for(let i=0;i<PROMOTIONS.length;i++){const promotion=PR
  OMOTIONS[i];moves.push({color:color,from:from,to:to,piece:piece,captured:captured,promotion:promotion,flags:flags|BITS.PR
  OMOTION})}}class Chess{_board=new Array(128);_turn="w";_kings={w:-1,b:-1};_epSquare=-1;_halfMoves=0;_moveNumber=0;_histor
  y=[];_castling={w:0,b:0};_hash=0n;_positionCount=new Map;constructor(fen=DEFAULT_POSITION,{skipValidation:skipValidation=
  !1}={}){this.load(fen,{skipValidation:skipValidation})}clear(){this._board=new Array(128),this._kings={w:-1,b:-1},this._t
  urn="w",this._castling={w:0,b:0},this._epSquare=-1,this._halfMoves=0,this._moveNumber=1,this._history=[],this._hash=this.
  _computeHash(),this._positionCount=new Map}load(fen,{skipValidation:skipValidation=!1}={}){let tokens=fen.split(/\\s+/);i
  f(tokens.length>=2&&tokens.length<6){const adjustments=["-","-","0","1"];fen=tokens.concat(adjustments.slice(-(6-tokens.l
  ength))).join(" ")}if(tokens=fen.split(/\\s+/),!skipValidation){const{ok:ok,error:error}=function validateFen(fen){const
  tokens=fen.split(/\\s+/);if(6!=tokens.length)return{ok:!1,error:"Invalid FEN: must contain six space-delimited fields"};
  const moveNumber=parseInt(tokens[5],10);if(isNaN(moveNumber)||moveNumber<=0)return{ok:!1,error:"Invalid FEN: move number
  must be a positive integer"};const halfMoves=parseInt(tokens[4],10);if(isNaN(halfMoves)||halfMoves<0)return{ok:!1,error:"
  Invalid FEN: half move counter number must be a non-negative integer"};if(!/^(-|[abcdefgh][36])$/.test(tokens[3]))return{
  ok:!1,error:"Invalid FEN: en-passant square is invalid"};if(/[^kKqQ-]/.test(tokens[2]))return{ok:!1,error:"Invalid FEN: c
```

```
astling availability is invalid"};if(!/^(w|b)$/.test(tokens[1]))return{ok:!1,error:"Invalid FEN: side-to-move is invalid"
};const rows=tokens[0].split("/");if(8!==rows.length)return{ok:!1,error:"Invalid FEN: piece data does not contain 8 \'/\'
-delimited rows"};for(let i=0;i<rows.length;i++){let sumFields=0,previousWasNumber=!1;for(let k=0;k<rows[i].length;k++)if
(isDigit(rows[i][k])){if(previousWasNumber)return{ok:!1,error:"Invalid FEN: piece data is invalid (consecutive number)"};
sumFields+=parseInt(rows[i][k],10),previousWasNumber=!0}else{if(!/^[prnbqkPRNBQK]$/.test(rows[i][k]))return{ok:!1,error:"
Invalid FEN: piece data is invalid (invalid piece)"};sumFields+=1,previousWasNumber=!1}if(8!==sumFields)return{ok:!1,erro
r:"Invalid FEN: piece data is invalid (too many squares in rank)"}}if("3"===tokens[3][1]&&"w"===tokens[1]||"6"===tokens[3][1
]&&"b"===tokens[1])return{ok:!1,error:"Invalid FEN: illegal en-passant square"};const kings=[{color:"white",regex:/K/g},{c
olor:"black",regex:/k/g}];for(const{color:color,regex:regex}of kings){if(!regex.test(tokens[0]))return{ok:!1,error:`Inval
id FEN: missing ${color} king`};if((tokens[0].match(regex)||[]).length>1)return{ok:!1,error:`Invalid FEN: too many ${colo
r} kings`}}return Array.from(rows[0]+rows[7]).some(char=>"P"===char.toUpperCase())?{ok:!1,error:"Invalid FEN: some pawns
are on the edge rows"}:{ok:!0}}(fen);if(!ok)throw new Error(error)}const position=tokens[0];let square=0;this.clear();for
(let i=0;i<position.length;i++){const piece=position.charAt(i);if("/"===piece)square+=8;else if(isDigit(piece))square+=pa
rseInt(piece,10);else{const color=piece<"a"?"w":"b";this._put({type:piece.toLowerCase(),color:color},algebraic(square)),s
quare++}}this._turn=tokens[1],tokens[2].indexOf("K")>-1&&(this._castling.w|=BITS.KSIDE_CASTLE),tokens[2].indexOf("Q")>-1&
&(this._castling.w|=BITS.QSIDE_CASTLE),tokens[2].indexOf("k")>-1&&(this._castling.b|=BITS.KSIDE_CASTLE),tokens[2].indexOf
("q")>-1&&(this._castling.b|=BITS.QSIDE_CASTLE),this._epSquare="-"===tokens[3]?-1:Ox88[tokens[3]],this._halfMoves=parseIn
t(tokens[4],10),this._moveNumber=parseInt(tokens[5],10),this._hash=this._computeHash(),this._incPositionCount()}fen({forc
eEnpassantSquare:forceEnpassantSquare=!1}={}){let empty=0,fen="";for(let i=Ox88.a8;i<=Ox88.h1;i++){if(this._board[i]){emp
ty>0&&(fen+=empty,empty=0);const{color:color,type:piece}=this._board[i];fen+="w"===color?piece.toUpperCase():piece.toLowe
rCase()}else empty++;i+1&136&&(empty>0&&(fen+=empty),i!==Ox88.h1&&(fen+="/"),empty=0,i+=8)}let castling="";this._castling
.w&BITS.KSIDE_CASTLE&&(castling+="K"),this._castling.w&BITS.QSIDE_CASTLE&&(castling+="Q"),this._castling.b&BITS.KSIDE_CAS
TLE&&(castling+="k"),this._castling.b&BITS.QSIDE_CASTLE&&(castling+="q"),castling=castling||"-";let epSquare="-";if(-1!==
this._epSquare)if(forceEnpassantSquare)epSquare=algebraic(this._epSquare);else{const bigPawnSquare=this._epSquare+("w"===
this._turn?16:-16),squares=[bigPawnSquare+1,bigPawnSquare-1];for(const square of squares){if(136&square)continue;const co
lor=this._turn;if(this._board[square]?.color===color&&"p"===this._board[square]?.type){this._makeMove({color:color,from:s
quare,to:this._epSquare,piece:"p",captured:"p",flags:BITS.EP_CAPTURE});const isLegal=!this._isKingAttacked(color);if(this
._undoMove(),isLegal){epSquare=algebraic(this._epSquare);break}}}}return[fen,this._turn,castling,epSquare,this._halfMoves
,this._moveNumber].join(" ")}_pieceKey(i){if(!this._board[i])return 0n;const{color:color,type:type}=this._board[i],typeIn
dex={p:0,n:1,b:2,r:3,q:4,k:5}[type];return PIECE_KEYS[{w:0,b:1}[color]][typeIndex][i]}_epKey(){return-1===this._epSquare?
0n:EP_KEYS[7&this._epSquare]}_castlingKey(){const index=this._castling.w>>5|this._castling.b>>3;return CASTLING_KEYS[inde
x]}_computeHash(){let hash=0n;for(let i=Ox88.a8;i<=Ox88.h1;i++)136&i?i+=7:this._board[i]&&(hash^=this._pieceKey(i));retur
n hash^=this._epKey(),hash^=this._castlingKey(),"b"===this._turn&&(hash^=SIDE_KEY),hash}reset(){this.load(DEFAULT_POSITIO
N)}get(square){return this._board[Ox88[square]]}_set(sq,piece){this._hash^=this._pieceKey(sq),this._board[sq]=piece,this.
_hash^=this._pieceKey(sq)}_put({type:type,color:color},square){if(-1===("pnbrqkPNBRQK".indexOf(type.toLowerCase())))return!
1;if(!(square in Ox88))return!1;const sq=Ox88[square];if("k"===type&&-1!==this._kings[color]&&this._kings[color]!==sq)return
!1;const currentPieceOnSquare=this._board[sq];return currentPieceOnSquare&&"k"===currentPieceOnSquare.type&&(this._kings[
currentPieceOnSquare.color]=-1),this._set(sq,{type:type,color:color}),"k"===type&&(this._kings[color]=sq),!0}_clear(sq){t
his._hash^=this._pieceKey(sq),delete this._board[sq]}_attacked(color,square,verbose){const attackers=[];for(let i=Ox88.a8
;i<=Ox88.h1;i++){if(136&i){i+=7;continue}if(void 0===this._board[i]||this._board[i].color!==color)continue;const piece=th
is._board[i],difference=i-square;if(0===difference)continue;const index=difference+119;if(ATTACKS[index]&PIECE_MASKS[piec
e.type]){if("p"===piece.type){if(difference>0&&"w"===piece.color||difference<=0&&"b"===piece.color){if(!verbose)return!0;
attackers.push(algebraic(i))}continue}if("n"===piece.type||"k"===piece.type){if(verbose){attackers.push(algebraic(i));con
tinue}return!0}const offset=RAYS[index];let j=i+offset,blocked=!1;for(;j!==square;){if(null!=this._board[j]){blocked=!0;b
reak}j+=offset}if(!blocked){if(verbose){attackers.push(algebraic(i));continue}return!0}}return!!verbose&&attackers}attac
kers(square,attackedBy){return attackedBy?this._attacked(attackedBy,Ox88[square],!0):this._attacked(this._turn,Ox88[squar
e],!0)}_isKingAttacked(color){const square=this._kings[color];return-1!==square&&this._attacked(swapColor(color),square)}
hash(){return this._hash.toString(16)}isAttacked(square,attackedBy){return this._attacked(attackedBy,Ox88[square])}isChec
k(){return this._isKingAttacked(this._turn)}inCheck(){return this.isCheck()}isCheckmate(){return this.isCheck()&&0===this
._moves().length}isStalemate(){return!this.isCheck()&&0===this._moves().length}isInsufficientMaterial(){const pieces={b:0
,n:0,r:0,q:0,k:0,p:0},bishops=[];let numPieces=0,squareColor=0;for(let i=Ox88.a8;i<=Ox88.h1;i++){if(squareColor=(squareCo
lor+1)%2,136&i){i+=7;continue}const piece=this._board[i];piece&&(pieces[piece.type]=piece.type in pieces?pieces[piece.typ
e]+1:1,"b"===piece.type&&bishops.push(squareColor),numPieces++)}if(2===numPieces)return!0;if(3===numPieces&&(1===pieces.b
||1===pieces.n))return!0;if(numPieces===pieces.b+2){let sum=0;const len=bishops.length;for(let i=0;i<len;i++)sum+=bishops
[i];if(0===sum||sum===len)return!0}return!1}isThreefoldRepetition(){return this._getPositionCount(this._hash)>=3}isDrawBy
FiftyMoves(){return this._halfMoves>=100}isDraw(){return this.isDrawByFiftyMoves()||this.isStalemate()||this.isInsufficie
ntMaterial()||this.isThreefoldRepetition()}isGameOver(){return this.isCheckmate()||this.isDraw()}moves({verbose:verbose=!
1,square:square,piece:piece}={}){const moves=this._moves({square:square,piece:piece});return verbose?moves.map(move=>new
Move(this,move)):moves.map(move=>this._moveToSan(move,moves))}_moves({legal:legal=!0,piece:piece,square:square}={}){const
forSquare=square?square.toLowerCase():void 0,forPiece=piece?.toLowerCase(),moves=[],us=this._turn,them=swapColor(us);let
firstSquare=Ox88.a8,lastSquare=Ox88.h1,singleSquare=!1;if(forSquare){if(!(forSquare in Ox88))return[];firstSquare=lastSq
uare=Ox88[forSquare],singleSquare=!0}for(let from=firstSquare;from<=lastSquare;from++){if(136&from){from+=7;continue}if(!
this._board[from]||this._board[from].color===them)continue;const{type:type}=this._board[from];let to;if("p"===type){if(fo
rPiece&&forPiece!==type)continue;to=from+PAWN_OFFSETS[us][0],this._board[to]||(addMove(moves,us,from,to,"p"),to=from+PAWN
_OFFSETS[us][1],SECOND_RANK[us]===rank(from)&&this._board[to]||addMove(moves,us,from,to,"p",void 0,BITS.BIG_PAWN));for(le
t j=2;j<4;j++)to=from+PAWN_OFFSETS[us][j],136&to||(this._board[to]?.color===them?addMove(moves,us,from,to,"p",this._board
[to].type,BITS.CAPTURE):to===this._epSquare&&addMove(moves,us,from,to,"p","p",BITS.EP_CAPTURE))}else{if(forPiece&&forPiec
e!==type)continue;for(let j=0,len=PIECE_OFFSETS[type].length;j<len;j++){const offset=PIECE_OFFSETS[type][j];for(to=from;t
o+=offset,!(136&to);){if(this._board[to]){if(this._board[to].color===us)break;addMove(moves,us,from,to,type,this._board[t
o].type,BITS.CAPTURE);break}if(addMove(moves,us,from,to,type),"n"===type||"k"===type)break}}}}if(!(void 0!==forPiece&&"k"
!==forPiece||singleSquare&&lastSquare!==this._kings[us])){if(this._castling[us]&BITS.KSIDE_CASTLE){const castlingFrom=thi
```

```
s._kings[us],castlingTo=castlingFrom+2;this._board[castlingFrom+1]||this._board[castlingTo]||this._attacked(them,this._kings[us])||this._attacked(them,castlingFrom+1)||this._attacked(them,castlingTo)||addMove(moves,us,this._kings[us],castlingTo,"k",void 0,BITS.KSIDE_CASTLE)}if(this._castling[us]&BITS.QSIDE_CASTLE){const castlingFrom=this._kings[us],castlingTo=castlingFrom-2;this._board[castlingFrom-1]||this._board[castlingFrom-2]||this._board[castlingFrom-3]||this._attacked(them,this._kings[us])||this._attacked(them,castlingFrom-1)||this._attacked(them,castlingTo)||addMove(moves,us,this._kings[us],castlingTo,"k",void 0,BITS.QSIDE_CASTLE)}}if(!legal||-1===this._kings[us])return moves;const legalMoves=[];for(let i=0,len=moves.length;i<len;i++)this._makeMove(moves[i]),this._isKingAttacked(us)||legalMoves.push(moves[i]),this._undoMove();return legalMoves}move(move){let moveObj=null;const moves=this._moves();for(let i=0,len=moves.length;i<len;i++)if(move.from===algebraic(moves[i].from)&&move.to===algebraic(moves[i].to)&&(!("promotion"in moves[i])||move.promotion===moves[i].promotion)){moveObj=moves[i];break}if(!moveObj)throw new Error(`Invalid move: ${JSON.stringify(move)}`);const prettyMove=new Move(this,moveObj);return this._makeMove(moveObj),this._incPositionCount(),prettyMove}_push(move){this._history.push({move:move,kings:{b:this._kings.b,w:this._kings.w},turn:this._turn,castling:{b:this._castling.b,w:this._castling.w},epSquare:this._epSquare,halfMoves:this._halfMoves,moveNumber:this._moveNumber})}_movePiece(from,to){this._hash^=this._pieceKey(from),this._board[to]=this._board[from],delete this._board[from],this._hash^=this._pieceKey(to)}_makeMove(move){const us=this._turn,them=swapColor(us);if(this._push(move),move.flags&BITS.NULL_MOVE)return"b"===us&&this._moveNumber++,this._halfMoves++,this._turn=them,void(this._epSquare=-1);if(this._hash^=this._epKey(),this._hash^=this._castlingKey(),move.captured&&(this._hash^=this._pieceKey(move.to)),this._movePiece(move.from,move.to),move.flags&BITS.EP_CAPTURE&&("b"===this._turn?this._clear(move.to-16):this._clear(move.to+16)),move.promotion&&(this._clear(move.to),this._set(move.to,{type:move.promotion,color:us})),"k"===this._board[move.to].type){if(this._kings[us]=move.to,move.flags&BITS.KSIDE_CASTLE){const castlingTo=move.to-1,castlingFrom=move.to+1;this._movePiece(castlingFrom,castlingTo)}else if(move.flags&BITS.QSIDE_CASTLE){const castlingTo=move.to+1,castlingFrom=move.to-2;this._movePiece(castlingFrom,castlingTo)}this._castling[us]=0}if(this._castling[us])for(let i=0,len=ROOKS[us].length;i<len;i++)if(move.from===ROOKS[us][i].square&&this._castling[us]&ROOKS[us][i].flag){this._castling[us]^=ROOKS[us][i].flag;break}if(this._castling[them])for(let i=0,len=ROOKS[them].length;i<len;i++)if(move.to===ROOKS[them][i].square&&this._castling[them]&ROOKS[them][i].flag){this._castling[them]^=ROOKS[them][i].flag;break}if(this._hash^=this._castlingKey(),move.flags&BITS.BIG_PAWN){let epSquare;epSquare="b"===us?move.to-16:move.to+16,(move.to-1&136||"p"!==this._board[move.to-1]?.type||this._board[move.to-1]?.color!==them)&&(move.to+1&136||"p"!==this._board[move.to+1]?.type||this._board[move.to+1]?.color!==them)?this._epSquare=-1:(this._epSquare=epSquare,this._hash^=this._epKey())}else this._epSquare=-1;"p"===move.piece||move.flags&(BITS.CAPTURE|BITS.EP_CAPTURE)?this._halfMoves=0:this._halfMoves++,"b"===us&&this._moveNumber++,this._turn=them,this._hash^=SIDE_KEY}undo(){const hash=this._hash,move=this._undoMove();if(move){const prettyMove=new Move(this,move);return this._decPositionCount(hash),prettyMove}return null}_undoMove(){const old=this._history.pop();if(void 0===old)return null;this._hash^=this._epKey(),this._hash^=this._castlingKey();const move=old.move;this._kings=old.kings,this._turn=old.turn,this._castling=old.castling,this._epSquare=old.epSquare,this._halfMoves=old.halfMoves,this._moveNumber=old.moveNumber,this._hash^=this._epKey(),this._hash^=this._castlingKey(),this._hash^=SIDE_KEY;const us=this._turn,them=swapColor(us);if(move.flags&BITS.NULL_MOVE)return move;if(this._movePiece(move.to,move.from),move.piece&&(this._clear(move.from),this._set(move.from,{type:move.piece,color:us})),move.captured)if(move.flags&BITS.EP_CAPTURE){let index;index="b"===us?move.to-16:move.to+16,this._set(index,{type:"p",color:them})}else this._set(move.to,{type:move.captured,color:them});if(move.flags&(BITS.KSIDE_CASTLE|BITS.QSIDE_CASTLE)){let castlingTo,castlingFrom;move.flags&BITS.KSIDE_CASTLE?(castlingTo=move.to+1,castlingFrom=move.to-1):(castlingTo=move.to-2,castlingFrom=move.to+1),this._movePiece(castlingFrom,castlingTo)}return move}_moveToSan(move,moves){let output="";if(move.flags&BITS.KSIDE_CASTLE)output="O-O";else if(move.flags&BITS.QSIDE_CASTLE)output="O-O-O";else{if("p"!==move.piece){const disambiguator=function getDisambiguator(move,moves){const from=move.from,to=move.to,piece=move.piece;let ambiguities=0,sameRank=0,sameFile=0;for(let i=0,len=moves.length;i<len;i++){const ambigFrom=moves[i].from,ambigTo=moves[i].to;piece===moves[i].piece&&from!==ambigFrom&&to===ambigTo&&(ambiguities++,rank(from)===rank(ambigFrom)&&sameRank++,file(from)===file(ambigFrom)&&sameFile++)}return ambiguities>0?sameRank>0&&sameFile>0?algebraic(from):sameFile>0?algebraic(from).charAt(1):algebraic(from).charAt(0):""}(move,moves);output+=move.piece.toUpperCase()+disambiguator}move.flags&(BITS.CAPTURE|BITS.EP_CAPTURE)&&("p"===move.piece&&(output+=algebraic(move.from)[0]),output+="x"),output+=algebraic(move.to),move.promotion&&(output+="="+move.promotion.toUpperCase())}return this._makeMove(move),this.isCheck()&&(this.isCheckmate()?output+="#":output+="+"),this._undoMove(),output}turn(){return this._turn}board(){const output=[];let row=[];for(let i=0x88.a8;i<=0x88.h1;i++)null===this._board[i]?row.push(null):row.push({square:algebraic(i),type:this._board[i].type,color:this._board[i].color}),i+1&136&&(output.push(row),row=[],i+=8);return output}squareColor(square){if(square in 0x88){const sq=0x88[square];return(rank(sq)+file(sq))%2==0?"light":"dark"}return null}history({verbose:verbose=!1}={}){const reversedHistory=[],moveHistory=[];for(;this._history.length>0;)reversedHistory.push(this._undoMove());for(;;){const move=reversedHistory.pop();if(!move)break;verbose?moveHistory.push(new Move(this,move)):moveHistory.push(this._moveToSan(move,this._moves())),this._makeMove(move)}return moveHistory}_getPositionCount(hash){return this._positionCount.get(hash)??0}_incPositionCount(){this._positionCount.set(this._hash,(this._positionCount.get(this._hash)??0)+1)}_decPositionCount(hash){const currentCount=this._positionCount.get(hash)??0;1===currentCount?this._positionCount.delete(hash):this._positionCount.set(hash,currentCount-1)}moveNumber(){return this._moveNumber}}const PIECE_VALUES={p:100,n:320,b:330,r:500,q:900,k:0};function searchRoot(chess,depth,ctx,previousBestMove){const candidates=[],legalMoves=orderMoves(chess.moves({verbose:!0}),previousBestMove);for(const move of legalMoves){checkTime(ctx),chess.move(toMoveCommand(move));const score=-negamax(chess,depth-1,-1e7,1e7,1,ctx);chess.undo(),candidates.push({move:toMoveCommand(move),san:move.san,score:score})}if(0===candidates.length)throw new SearchTimeout;const principal=function selectBestRootMove(candidates){return candidates.reduce((currentBest,candidate)=>candidate.score>currentBest.score?candidate:currentBest)}(candidates),selection=function selectRootMove(candidates,settings,best){const level=settings.randomness/30,windowCp=20+150*level,floorScore=best.score-windowCp,ticketPower=4-1.5*level;if(function shouldForceBestMove(settings,best){return 0===settings.randomness||function isMateProtectedScore(score){return Math.abs(score)>=99e4}(best.score)||"hard"===settings.preset&&best.score<0}(settings,best))return{selected:best,debug:createDeterministicDebug(candidates,best,best,windowCp,floorScore,ticketPower)};const weightedCandidates=candidates.map(candidate=>({candidate:candidate,tickets:(Math.max(0,candidate.score-floorScore)/windowCp)**ticketPower})).filter(({tickets:tickets})=>tickets>0),totalTickets=weighedCandidates.reduce((total,{tickets:tickets})=>total+tickets,0);if(totalTickets<=0)return{selected:best,debug:createDeterministicDebug(candidates,best,best,windowCp,floorScore,ticketPower)};let draw=Math.random()*totalTickets,selected=best;for(const{candidate:candidate,tickets:tickets}of weightedCandidates)if(draw-=tickets,draw<=0){selected=candidate;break}return{selected:selected,debug:createWeightedDebug(candidates,best,selected,weightedCandidates,windowCp,floorScore,ticketPow
```

```
er,totalTickets)}}(candidates,ctx.settings,principal);return{selected:selection.selected,principal:principal,debug:select
ion.debug}}function shouldStopAfterSoftLimit(chess,rootMoves,result,previousPrincipal,depth,elapsedMs,settings){if("hard"
!==settings.preset||void 0===settings.softTimeLimitMs||elapsedMs<settings.softTimeLimitMs||depth<2)return!1;if(chess.isCh
eck()||result.principal.score<0)return!1;if(null===previousPrincipal||!isSameMoveCommand(previousPrincipal.move,result.pr
incipal.move)||Math.abs(previousPrincipal.score-result.principal.score)>90)return!1;const scoreGap=function rootScoreGap(
debug){const[bestCandidate,secondCandidate]=debug.candidates;return bestCandidate&&secondCandidate?bestCandidate.score-se
condCandidate.score:1e7}(result.debug);return!(scoreGap<120||function tacticalMoveRatio(moves){return 0===moves.length?0:
moves.filter(isTacticalMove).length/moves.length}(rootMoves)>=.35&&scoreGap<220)}function negamax(chess,depth,alpha,beta,
ply,ctx){if(checkTime(ctx),chess.isCheckmate()||chess.isDraw())return terminalScore(chess,ply);if(depth<=0)return ctx.set
tings.quiescence?quiescence(chess,alpha,beta,ply,ctx):evaluatePosition(chess);let bestScore=-1e7,currentAlpha=alpha;const
legalMoves=orderMoves(chess.moves({verbose:!0}),null);for(const move of legalMoves){chess.move(toMoveCommand(move));cons
t score=-negamax(chess,depth-1,-beta,-currentAlpha,ply+1,ctx);if(chess.undo(),bestScore=Math.max(bestScore,score),current
Alpha=Math.max(currentAlpha,score),currentAlpha>=beta)break}return bestScore}function quiescence(chess,alpha,beta,ply,ctx
){if(checkTime(ctx),chess.isCheckmate()||chess.isDraw())return terminalScore(chess,ply);let currentAlpha=alpha;const stan
dPat=evaluatePosition(chess);if(standPat>=beta)return beta;if(currentAlpha=Math.max(currentAlpha,standPat),ply>=6)return
currentAlpha;const legalMoves=orderMoves(chess.moves({verbose:!0}),null),tacticalMoves=chess.isCheck()?legalMoves:legalMo
ves.filter(isTacticalMove);for(const move of tacticalMoves){chess.move(toMoveCommand(move));const score=-quiescence(chess
,-beta,-currentAlpha,ply+1,ctx);if(chess.undo(),score>=beta)return beta;currentAlpha=Math.max(currentAlpha,score)}return
currentAlpha}function evaluatePosition(chess){const board=chess.board(),pawns=[],bishops={w:0,b:0},kings={};let whiteScor
e=0,blackScore=0,nonPawnMaterial=0;for(const row of board)for(const pieceOnSquare of row){if(null===pieceOnSquare)continu
e;const{color:color,type:type,square:square}=pieceOnSquare,pieceScore=PIECE_VALUES[type]+pieceSquareScore(type,color,squa
re);"w"===color?whiteScore+=pieceScore:blackScore+=pieceScore,"p"===type?pawns.push({color:color,file:fileIndex(square),r
ank:rankNumber(square)}):"k"!==type&&(nonPawnMaterial+=PIECE_VALUES[type]),"b"===type&&(bishops[color]+=1),"k"===type&&(k
ings[color]=square)}whiteScore+=bishops.w>=2?35:0,blackScore+=bishops.b>=2?35:0,whiteScore+=pawnStructureScore(pawns,"w")
,blackScore+=pawnStructureScore(pawns,"b"),whiteScore+=developmentScore(board,"w"),blackScore+=developmentScore(board,"b"
),whiteScore+=kingSafetyScore(kings.w,pawns,"w",nonPawnMaterial),blackScore+=kingSafetyScore(kings.b,pawns,"b",nonPawnMat
erial);let score=(whiteScore-blackScore)*("w"===chess.turn()?1:-1);return score+=2*chess.moves().length,chess.isCheck()&&
(score-=45),score}function pieceSquareScore(type,color,square){const file2=fileIndex(square),relativeRank="w"===color?ran
kNumber(square):9-rankNumber(square),centerDistance=Math.abs(file2-3.5)+Math.abs(rankNumber(square)-4.5);switch(type){cas
e"p":return 7*relativeRank+(file2>=2&&file2<=5?8:0);case"n":return 42-12*centerDistance-(1===relativeRank?12:0);case"b":r
eturn 30-7*centerDistance;case"r":return(relativeRank>=7?18:0)+(0===file2||7===file2?4:0);case"q":return 14-3*centerDista
nce;case"k":return relativeRank>=7?-18:0}}function pawnStructureScore(pawns,color){const ownPawns=pawns.filter(pawn=>pawn
.color===color),fileCounts=Array.from({length:8},()=>0);let score=0;for(const pawn of ownPawns)fileCounts[pawn.file]+=1;f
or(const pawn of ownPawns){const relativeRank="w"===color?pawn.rank:9-pawn.rank;fileCounts[pawn.file]>1&&(score-=14),0===
(fileCounts[pawn.file-1]??0)&&0===(fileCounts[pawn.file+1]??0)&&(score-=10),isPassedPawn(pawn,pawns)&&(score+=(relativeRa
nk-1)*(relativeRank-1)*9)}return score}function isPassedPawn(pawn,pawns){const enemyColor=function oppositeColor(color){r
eturn"w"===color?"b":"w"}(pawn.color);return!pawns.some(candidate=>!(candidate.color!==enemyColor||Math.abs(candidate.fil
e-pawn.file)>1)&&("w"===pawn.color?candidate.rank>pawn.rank:candidate.rank<pawn.rank))}function developmentScore(board,co
lor){const homeRank="w"===color?"1":"8",knightHomeSquares="w"===color?new Set(["b1","g1"]):new Set(["b8","g8"]),bishopHom
eSquares="w"===color?new Set(["c1","f1"]):new Set(["c8","f8"]);let score=0;for(const row of board)for(const piece of row)
null!==piece&&piece.color===color&&("n"!==piece.type||knightHomeSquares.has(piece.square)||(score+=18),"b"!==piece.type||
bishopHomeSquares.has(piece.square)||(score+=16),"q"===piece.type&&piece.square.endsWith(homeRank)&&hasMovedMinorPieces(b
oard,color)&&(score-=12));return score}function hasMovedMinorPieces(board,color){const homeSquares="w"===color?new Set(["
b1","c1","f1","g1"]):new Set(["b8","c8","f8","g8"]);for(const row of board)for(const piece of row)if(null!==piece&&piece.
color===color&&("n"===piece.type||"b"===piece.type)&&homeSquares.has(piece.square))return!1;return!0}function kingSafetyS
core(kingSquare,pawns,color,nonPawnMaterial){if(!kingSquare||nonPawnMaterial<1800)return 0;const kingFile=fileIndex(kingS
quare),kingRank=rankNumber(kingSquare),shieldRank="w"===color?kingRank+1:kingRank-1;let score=2===kingFile||6===kingFile?
28:-18;for(let file2=kingFile-1;file2<=kingFile+1;file2+=1)file2<0||file2>=8||(score+=pawns.some(pawn=>pawn.color===color
&&pawn.file===file2&&pawn.rank===shieldRank)?14:-10);return score}function terminalScore(chess,ply){return chess.isCheckm
ate()?-1e6+ply:chess.isDraw()?0:evaluatePosition(chess)}function orderMoves(moves,preferredMove){return[...moves].sort((l
eft,right)=>moveOrderingScore(right,preferredMove)-moveOrderingScore(left,preferredMove))}function moveOrderingScore(move
,preferredMove){let score=0;return preferredMove&&function isSameMove(move,command){return move.from===command.from&&move
.to===command.to&&(move.promotion??void 0)===(command.promotion??void 0)}(move,preferredMove)&&(score+=2e6),move.san.incl
udes("#")?score+=15e5:move.san.includes("+")&&(score+=16e3),move.isCapture()&&(score+=12*(move.captured?PIECE_VALUES[move
.captured]:PIECE_VALUES.p)-PIECE_VALUES[move.piece]),move.isPromotion()&&move.promotion&&(score+=PIECE_VALUES[move.promot
ion]+8e3),(move.isKingsideCastle()||move.isQueensideCastle())&&(score+=1500),score+=2*pieceSquareScore(move.piece,move.co
lor,move.to),score}function checkTime(ctx){if(ctx.nodes+=1,ctx.nodes%2048!=0)return;const now=Date.now();if(now-ctx.lastP
rogressAt>=100&&publishSearchProgress(ctx,ctx.progressState,now-ctx.startTime,now),now>=ctx.deadline)throw new SearchTime
out}function publishSearchProgress(ctx,progressState,elapsedMs=Date.now()-ctx.startTime,progressAt=Date.now()){ctx.progre
ssState=progressState,ctx.lastProgressAt=progressAt,ctx.onProgress?.({kind:"progress",id:ctx.requestId,move:progressState
.move,depthReached:progressState.depthReached,nodes:ctx.nodes,elapsedMs:elapsedMs,score:progressState.score,debug:progres
sState.debug})}function isTacticalMove(move){return move.isCapture()||move.isPromotion()||move.san.includes("+")||move.sa
n.includes("#")}function createDeterministicDebug(candidates,principal,selected,windowCp,floorScore,ticketPower){return{c
andidates:sortCandidatesByScore(candidates).map(candidate=>({move:candidate.move,san:candidate.san,score:candidate.score,
tickets:isSameMoveCommand(candidate.move,selected.move)?1:0,probability:isSameMoveCommand(candidate.move,selected.move)?1
:0,selected:isSameMoveCommand(candidate.move,selected.move),principal:isSameMoveCommand(candidate.move,principal.move)}))
,bestScore:principal.score,windowCp:windowCp,floorScore:floorScore,ticketPower:ticketPower,totalTickets:1}}function creat
eWeightedDebug(candidates,principal,selected,weightedCandidates,windowCp,floorScore,ticketPower,totalTickets){return{cand
idates:sortCandidatesByScore(candidates).map(candidate=>{const weightedCandidate=weightedCandidates.find(({candidate:weig
hted})=>isSameMoveCommand(weighted.move,candidate.move)),tickets=weightedCandidate?.tickets??0;return{move:candidate.move
,san:candidate.san,score:candidate.score,tickets:tickets,probability:totalTickets>0?tickets/totalTickets:0,selected:isSam
```

```
    | ↪ eMoveCommand(candidate.move,selected.move),principal:isSameMoveCommand(candidate.move,principal.move)}}),bestScore:princi
    | ↪ pal.score,windowCp:windowCp,floorScore:floorScore,ticketPower:ticketPower,totalTickets:totalTickets}}function·sortCandida
    | ↪ tesByScore(candidates){return[...candidates].sort((left,right)⇒right.score-left.score)}function·toMoveCommand(move){retu
    | ↪ rn·move.promotion?{from:move.from,to:move.to,promotion:move.promotion}:{from:move.from,to:move.to}}function·isSameMoveCom
    | ↪ mand(left,right){return·left.from≡=right.from&&left.to≡=right.to&&(left.promotion??void·0)≡=(right.promotion??void·0)}
    | ↪ function·fileIndex(square){return·square.charCodeAt(0)-97}function·rankNumber(square){return·Number(square[1])}function·c
    | ↪ lamp(value,min,max){return·Math.min(max,Math.max(min,value))}class·SearchTimeout·extends·Error{}const·workerScope=globalT
    | ↪ his;workerScope.addEventListener("message",event⇒{const·response=function·searchBestMove(request,onProgress){const·chess
    | ↪ =new·Chess(request.fen),settings=function·normalizeAiSettings(settings){const·timeLimitMs=clamp(Math.round(settings.timeL
    | ↪ imitMs),500,1e4),softTimeLimitMs=void·0≡=settings.softTimeLimitMs?void·0:Math.min(timeLimitMs,clamp(Math.round(settings.
    | ↪ softTimeLimitMs),500,1e4));return{preset:settings.preset,maxDepth:(depth=settings.maxDepth,clamp(Math.round(depth),1,5)),
    | ↪ timeLimitMs:timeLimitMs,...void·0≡=softTimeLimitMs?{}:{softTimeLimitMs:softTimeLimitMs},randomness:30,quiescence:!0};var
    | ↪ ·depth}(request.settings),legalMoves=chess.moves({verbose:!0}),startTime=Date.now(),ctx={requestId:request.id,settings:se
    | ↪ ttings,deadline:startTime+settings.timeLimitMs,startTime:startTime,nodes:0,onProgress:onProgress,lastProgressAt:startTime
    | ↪ ,progressState:{move:null,depthReached:0,score:0}};if(0≡=legalMoves.length)return{id:request.id,move:null,depthReached:0
    | ↪ ,nodes:0,elapsedMs:0,score:terminalScore(chess,0)};let·debug,best=null,previousBestMove=null,previousPrincipal=null,depth
    | ↪ Reached=0;for(let·depth=1;depth⩽settings.maxDepth;depth+=1){try{const·result=searchRoot(chess,depth,ctx,previousBestMove
    | ↪ ),elapsedMs=Date.now()-startTime;if(best=result.selected,debug=result.debug,previousBestMove=result.principal.move,depthR
    | ↪ eached=depth,publishSearchProgress(ctx,{move:best.move,depthReached:depthReached,score:best.score,debug:debug},elapsedMs)
    | ↪ ,shouldStopAfterSoftLimit(chess,legalMoves,result,previousPrincipal,depth,elapsedMs,settings))break;previousPrincipal=res
    | ↪ ult.principal}catch(error){if(error·instanceof·SearchTimeout)break;throw·error}if(Date.now()⩾ctx.deadline)break}return·b
    | ↪ est??=function·chooseFallbackMove(chess){const[move]=orderMoves(chess.moves({verbose:!0}),null);if(!move)return{move:{fro
    | ↪ m:"a1",to:"a1"},score:terminalScore(chess,0)};chess.move(toMoveCommand(move));const·score=-evaluatePosition(chess);return
    | ↪ ·chess.undo(),{move:toMoveCommand(move),score:score}}(chess),{id:request.id,move:best.move,depthReached:depthReached,node
    | ↪ s:ctx.nodes,elapsedMs:Date.now()-startTime,score:best.score,debug:debug}}(event.data,progress⇒{workerScope.postMessage(p
    | ↪ rogress)});workerScope.postMessage({...response,kind:"result"})})}();\n',
951 | ··blob·=
952 | ····'undefined'·≠=·typeof·self·&&
953 | ····self.Blob·&&
954 | ····new·Blob(['(self.URL·||·self.webkitURL).revokeObjectURL(self.location.href);',·jsContent],·{
955 | ····type:·'text/javascript;charset=utf-8',
956 | ····});
957 | function·WorkerWrapper(options)·{
958 | ··let·objURL;
959 | ··try·{
960 | ····if·(((objURL·=·blob·&&·(self.URL·||·self.webkitURL).createObjectURL(blob)),·!objURL))·throw·'';
961 | ····const·worker·=·new·Worker(objURL,·{·name:·options?.name·});
962 | ····return·(
963 | ····worker.addEventListener('error',·()·⇒·{
964 | ····(self.URL·||·self.webkitURL).revokeObjectURL(objURL);
965 | ····}),
966 | ····worker
967 | ····);
968 | ··}·catch·(e)·{
969 | ····return·new·Worker('data:text/javascript;charset=utf-8,'·+·encodeURIComponent(jsContent),·{
970 | ····name:·options?.name,
971 | ····});
972 | ··}
973 | }
974 | const·BOARD_FILES·=·['a',·'b',·'c',·'d',·'e',·'f',·'g',·'h'],
975 | ··BLACK_FILES·=·[...BOARD_FILES].reverse(),
976 | ··WHITE_RANKS·=·[8,·7,·6,·5,·4,·3,·2,·1],
977 | ··BLACK_RANKS·=·[...WHITE_RANKS].reverse(),
978 | ··PROMOTION_PIECES·=·['q',·'r',·'b',·'n'],
979 | ··SQUARE_SET·=·new·Set([
980 | ····'a8',
981 | ····'b8',
982 | ····'c8',
983 | ····'d8',
984 | ····'e8',
985 | ····'f8',
986 | ····'g8',
987 | ····'h8',
988 | ····'a7',
989 | ····'b7',
990 | ····'c7',
991 | ····'d7',
992 | ····'e7',
993 | ····'f7',
994 | ····'g7',
995 | ····'h7',
```

```
    'a6',
    'b6',
    'c6',
    'd6',
    'e6',
    'f6',
    'g6',
    'h6',
    'a5',
    'b5',
    'c5',
    'd5',
    'e5',
    'f5',
    'g5',
    'h5',
    'a4',
    'b4',
    'c4',
    'd4',
    'e4',
    'f4',
    'g4',
    'h4',
    'a3',
    'b3',
    'c3',
    'd3',
    'e3',
    'f3',
    'g3',
    'h3',
    'a2',
    'b2',
    'c2',
    'd2',
    'e2',
    'f2',
    'g2',
    'h2',
    'a1',
    'b1',
    'c1',
    'd1',
    'e1',
    'f1',
    'g1',
    'h1',
  ]),
  CLOCK_PRESETS = {
    '1/1': { baseMs: 6e4, incrementMs: 1e3 },
    '3/2': { baseMs: 18e4, incrementMs: 2e3 },
    '5/3': { baseMs: 3e5, incrementMs: 3e3 },
    '10/5': { baseMs: 6e5, incrementMs: 5e3 },
    '15/10': { baseMs: 9e5, incrementMs: 1e4 },
  },
  PIECE_GLYPHS = {
    w: { p: '♙', n: '♘', b: '♗', r: '♖', q: '♕', k: '♔' },
    b: { p: '♟', n: '♞', b: '♝', r: '♜', q: '♛', k: '♚' },
  };
function requireElement(selector, root) {
  const element = root.querySelector(selector);
  if (!element) throw new Error(`Missing element: ${selector}`);
  return element;
}
function createCoordinate(type, label) {
  const coordinate = document.createElement('span');
  return (
    (coordinate.className = `coordinate coordinate-${type}`),
    (coordinate.textContent = label),
    coordinate
```

```
  );
}
function toSquare(file2, rank2) {
  return `${file2}${rank2}`;
}
function createClockState(preset) {
  const baseMs = CLOCK_PRESETS[preset].baseMs;
  return { w: baseMs, b: baseMs };
}
function colorName(color) {
  return 'w' === color ? '백' : '흑';
}
function presetName(preset) {
  switch (preset) {
    case 'easy':
      return '쉬움';
    case 'normal':
      return '보통';
    case 'hard':
      return '어려움';
  }
}
function formatSeconds(milliseconds) {
  return `${(milliseconds / 1e3).toFixed(milliseconds % 1e3 === 0 ? 0 : 2)}초`;
}
function formatDebugScore(score) {
  return Math.abs(score) >= 9e5
    ? score > 0
      ? '+mate-ish'
      : '-mate-ish'
    : score > 0
      ? `+${Math.round(score)}`
      : String(Math.round(score));
}
function oppositeColor(color) {
  return 'w' === color ? 'b' : 'w';
}
function getSquareLabel(square, piece) {
  return piece ? `${square} ${colorName(piece.color)} ${pieceName(piece.type)}` : `${square} 빈 칸`;
}
function pieceName(piece) {
  switch (piece) {
    case 'p':
      return '폰';
    case 'n':
      return '나이트';
    case 'b':
      return '비숍';
    case 'r':
      return '룩';
    case 'q':
      return '퀸';
    case 'k':
      return '킹';
  }
}
!(function createChessApp() {
  const root = requireElement('#chess-app', document),
    boardElement = requireElement('#chess-board', root),
    colorControlsElement = requireElement('#color-controls', root),
    aiControlsElement = requireElement('#ai-controls', root),
    statusElement = requireElement('#game-status', root),
    turnMetaElement = requireElement('#turn-meta', root),
    modeMetaElement = requireElement('#mode-meta', root),
    moveMetaElement = requireElement('#move-meta', root),
    aiCandidatePanelElement = requireElement('#ai-candidate-panel', root),
    aiCandidateMetaElement = requireElement('#ai-candidate-meta', root),
    aiInfoStatusElement = requireElement('#ai-info-status', root),
    aiInfoPresetElement = requireElement('#ai-info-preset', root),
    aiInfoHumanElement = requireElement('#ai-info-human', root),
    aiInfoDepthElement = requireElement('#ai-info-depth', root),
```

```
    aiInfoNodesElement = requireElement('#ai-info-nodes', root),
    aiInfoScoreElement = requireElement('#ai-info-score', root),
    aiCandidateListElement = requireElement('#ai-candidate-list', root),
    moveListElement = requireElement('#move-list', root),
    clockDisplayElement = requireElement('#clock-display', root),
    whiteClockElement = requireElement('#white-clock', root),
    blackClockElement = requireElement('#black-clock', root),
    whiteClockTimeElement = requireElement('#white-clock-time', root),
    blackClockTimeElement = requireElement('#black-clock-time', root),
    newGameButton = requireElement('#new-game-button', root),
    undoButton = requireElement('#undo-button', root),
    aiDepthInput = requireElement('#ai-depth-input', root),
    aiDepthValue = requireElement('#ai-depth-value', root),
    aiTimeInput = requireElement('#ai-time-input', root),
    aiTimeValue = requireElement('#ai-time-value', root),
    promotionOverlay = requireElement('#promotion-overlay', root),
    promotionChoices = requireElement('#promotion-choices', root),
    modeButtons = Array.from(root.querySelectorAll('[data-mode]')),
    colorButtons = Array.from(root.querySelectorAll('[data-color]')),
    presetButtons = Array.from(root.querySelectorAll('[data-preset]')),
    clockPresetButtons = Array.from(root.querySelectorAll('[data-clock-preset]')),
    game = new Chess();
  let mode = 'local',
    clockPreset = '10/5',
    clockMs = createClockState(clockPreset),
    activeClockColor = null,
    lastClockTick = Date.now(),
    clockTimerId = null,
    timedOutColor = null,
    humanColor = 'w',
    selectedSquare = null,
    legalMoves = [],
    pendingPromotion = null,
    aiThinking = !1,
    aiTimerId = null,
    aiWorker = null,
    aiRequestId = 0,
    activeAiRequestId = null,
    activeAiFen = null,
    aiSettings = { ...DEFAULT_AI_SETTINGS },
    aiProgress = null;
  function selectSquare(square) {
    ((selectedSquare = square),
      (legalMoves = game.moves({ verbose: !0, square: square })),
      render());
  }
  function playMove(moveCommand) {
    if (!commitMove(moveCommand)) return (clearSelection(), void render());
    ((pendingPromotion = null), clearSelection(), render(), scheduleAiIfNeeded());
  }
  function resetGame() {
    (cancelAiMove(),
      game.reset(),
      (pendingPromotion = null),
      clearSelection(),
      resetClockState(),
      restartClockForCurrentTurn(),
      render(),
      scheduleAiIfNeeded());
  }
  function scheduleAiIfNeeded() {
    'ai' !== mode ||
      game.isGameOver() ||
      null !== timedOutColor ||
      game.turn() === humanColor ||
      aiThinking ||
      pendingPromotion ||
      ((aiThinking = !0),
      (aiProgress = null),
      render(),
      (aiTimerId = window.setTimeout(() => {
```

```
((aiTimerId = null),
(function startAiSearch() {
aiWorker?.terminate();
const requestId = aiRequestId + 1,
fen = game.fen();
((aiRequestId = requestId),
(activeAiRequestId = requestId),
(activeAiFen = fen),
(aiWorker = new WorkerWrapper()),
aiWorker.addEventListener('message', (event) => {
!(function handleAiMessage(message) {
if (
message.id === activeAiRequestId &&
null !== activeAiFen &&
game.fen() === activeAiFen
)) {
if (((aiProgress = message), 'progress' === message.kind))
return (renderStatus(), void renderAiCandidateDebug());
((aiThinking = !1),
(activeAiRequestId = null),
(activeAiFen = null),
aiWorker?.terminate(),
(aiWorker = null),
message.move && commitMove(message.move),
clearSelection(),
render());
}
})(event.data);
}),
aiWorker.addEventListener('error', () => {
activeAiRequestId === requestId &&
((aiThinking = !1),
(activeAiRequestId = null),
(activeAiFen = null),
aiWorker?.terminate(),
(aiWorker = null),
render());
}),
aiWorker.postMessage({
id: requestId,
fen: fen,
settings: resolveAiSettingsForSearch(),
}));
})());
}, 180)));
}
function cancelAiMove() {
((aiRequestId += 1),
(activeAiRequestId = null),
(activeAiFen = null),
(aiProgress = null),
null !== aiTimerId && (window.clearTimeout(aiTimerId), (aiTimerId = null)),
aiWorker && (aiWorker.terminate(), (aiWorker = null)),
(aiThinking = !1));
}
function commitMove(moveCommand) {
const movingColor = game.turn();
if (!flushClockElapsed()) return !1;
try {
game.move(moveCommand);
} catch {
return !1;
}
return (
(clockMs[movingColor] += CLOCK_PRESETS[clockPreset].incrementMs),
restartClockForCurrentTurn(),
!0
);
}
function updateAiSettings(settings) {
((aiSettings = normalizeAiSettings(settings)),
```

```
    aiThinking && cancelAiMove(),
    render(),
    scheduleAiIfNeeded());
}
function resolveAiSettingsForSearch() {
  const settings = normalizeAiSettings(aiSettings);
  if ('hard' !== settings.preset) return settings;
  const aiColor = game.turn(),
    remainingMs = clockMs[aiColor],
    targetMs = remainingMs / 30 + 0.8 * CLOCK_PRESETS[clockPreset].incrementMs,
    remainingSearchBudgetMs = Math.max(500, remainingMs - 250),
    softTimeLimitMs = Math.min(Math.max(targetMs, 800), 6e3, remainingSearchBudgetMs),
    timeLimitMs = Math.min(Math.max(1.8 * softTimeLimitMs, 1200), 1e4, remainingSearchBudgetMs);
  return normalizeAiSettings({
    ...settings,
    softTimeLimitMs: softTimeLimitMs,
    timeLimitMs: timeLimitMs,
  });
}
function resetClockState() {
  ((clockMs = createClockState(clockPreset)),
    (timedOutColor = null),
    (activeClockColor = null),
    (lastClockTick = Date.now()));
}
function restartClockForCurrentTurn() {
  if ((clearClockTimer(), game.isGameOver() || null !== timedOutColor))
    return ((activeClockColor = null), void renderClocks());
  ((activeClockColor = game.turn()),
    (lastClockTick = Date.now()),
    (clockTimerId = window.setInterval(tickClock, 100)),
    renderClocks());
}
function clearClockTimer() {
  null !== clockTimerId && (window.clearInterval(clockTimerId), (clockTimerId = null));
}
function tickClock() {
  flushClockElapsed() && renderClocks();
}
function flushClockElapsed() {
  if (null === activeClockColor || null !== timedOutColor || game.isGameOver()) return !0;
  const now = Date.now(),
    elapsedMs = Math.max(0, now - lastClockTick);
  return (
    (lastClockTick = now),
    0 === elapsedMs ||
      ((clockMs[activeClockColor] = Math.max(0, clockMs[activeClockColor] - elapsedMs)),
      0 !== clockMs[activeClockColor] ||
        ((function handleClockTimeout(color) {
          ((timedOutColor = color),
            (activeClockColor = null),
            (clockMs[color] = 0),
            clearClockTimer(),
            cancelAiMove(),
            clearSelection(),
            render());
        })(activeClockColor),
        !1))
  );
}
function clearSelection() {
  ((selectedSquare = null), (legalMoves = []));
}
function render() {
  !(function renderControls() {
    (modeButtons.forEach((button) => {
      const isActive = button.dataset.mode === mode;
      (button.classList.toggle('is-active', isActive),
        button.setAttribute('aria-pressed', String(isActive)));
    }),
      colorButtons.forEach((button) => {
```

```
      const isActive = button.dataset.color === humanColor;
      (button.classList.toggle('is-active', isActive),
        button.setAttribute('aria-pressed', String(isActive)));
    }),
    (colorControlsElement.hidden = 'ai' !== mode),
    (aiControlsElement.hidden = 'ai' !== mode),
    (function renderAiSettingsControls() {
      (presetButtons.forEach((button) => {
        const isActive = button.dataset.preset === aiSettings.preset;
        (button.classList.toggle('is-active', isActive),
          button.setAttribute('aria-pressed', String(isActive)));
      }),
        (aiDepthInput.value = String(aiSettings.maxDepth)),
        (aiDepthValue.textContent = `${aiSettings.maxDepth}`),
        (aiTimeInput.value = String(aiSettings.timeLimitMs)),
        (aiTimeInput.disabled = 'hard' === aiSettings.preset),
        (aiTimeValue.textContent =
          'hard' === aiSettings.preset ? '자동' : `${formatSeconds(aiSettings.timeLimitMs)}`));
    })(),
    (function renderClockSettingsControls() {
      clockPresetButtons.forEach((button) => {
        const isActive = button.dataset.clockPreset === clockPreset;
        (button.classList.toggle('is-active', isActive),
          button.setAttribute('aria-pressed', String(isActive)));
      });
    })(),
    (undoButton.disabled = 0 === game.history().length));
})(),
  renderClocks(),
  (function renderBoard() {
    const fragment = document.createDocumentFragment(),
      orientation = getBoardOrientation(),
      files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
      ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
      lastMove = (function getLastMove() {
        return game.history({ verbose: !0 }).at(-1) ?? null;
      })();
    boardElement.classList.toggle('is-locked', !canUseBoard());
    for (const rank2 of ranks)
      for (const file2 of files) {
        const square = toSquare(file2, rank2),
          piece = game.get(square),
          button = document.createElement('button');
        if (
          ((button.type = 'button'),
          (button.className = getSquareClasses(square, lastMove)),
          (button.dataset.square = square),
          (button.disabled = !canUseBoard()),
          button.setAttribute('role', 'gridcell'),
          button.setAttribute('aria-label', getSquareLabel(square, piece)),
          file2 === files[0] && button.appendChild(createCoordinate('rank', String(rank2))),
          rank2 === ranks[ranks.length - 1] &&
            button.appendChild(createCoordinate('file', file2)),
          piece)
        ) {
          const pieceElement = document.createElement('span');
          ((pieceElement.className = 'piece'),
            (pieceElement.textContent = PIECE_GLYPHS[piece.color][piece.type]),
            button.appendChild(pieceElement));
        }
        fragment.appendChild(button);
      }
    boardElement.replaceChildren(fragment);
  })(),
  renderStatus(),
  renderAiCandidateDebug(),
  (function renderMoveList() {
    const history = game.history({ verbose: !0 }),
      fragment = document.createDocumentFragment();
    if (0 === history.length) {
      const empty = document.createElement('p');
```

```
        return (
          (empty.className = 'move-empty'),
          (empty.textContent = '아직 둔 수가 없습니다.'),
          fragment.appendChild(empty),
          void moveListElement.replaceChildren(fragment)
        );
      }
      for (let index = 0; index < history.length; index += 2) {
        const row = document.createElement('div'),
          moveNumber = document.createElement('span'),
          whiteMove = document.createElement('span'),
          blackMove = document.createElement('span');
        ((row.className = 'move-row'),
          (moveNumber.className = 'move-number'),
          (whiteMove.className = 'move-san'),
          (blackMove.className = 'move-san'),
          (moveNumber.textContent = `${Math.floor(index / 2) + 1}.`),
          (whiteMove.textContent = history[index]?.san ?? ''),
          (blackMove.textContent = history[index + 1]?.san ?? ''),
          row.append(moveNumber, whiteMove, blackMove),
          fragment.appendChild(row));
      }
      (moveListElement.replaceChildren(fragment),
        (moveListElement.scrollTop = moveListElement.scrollHeight));
    })(),
    (function renderPromotionDialog() {
      if (
        ((promotionOverlay.hidden = null === pendingPromotion),
        promotionChoices.replaceChildren(),
        pendingPromotion)
      )
        for (const piece of PROMOTION_PIECES) {
          const button = document.createElement('button');
          ((button.className = 'promotion-button'),
            (button.type = 'button'),
            (button.dataset.promotion = piece),
            button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
            (button.textContent = PIECE_GLYPHS[game.turn()][piece]),
            promotionChoices.appendChild(button));
        }
    })());
  }
  function renderClocks() {
    const bottomColor = getBoardOrientation(),
      topClockElement = 'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
      bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
    ((clockDisplayElement.children[0] === topClockElement &&
      clockDisplayElement.children[1] === bottomClockElement) ||
      clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
      renderClockPosition('w', bottomColor),
      renderClockPosition('b', bottomColor),
      renderClock('w', whiteClockElement, whiteClockTimeElement),
      renderClock('b', blackClockElement, blackClockTimeElement));
  }
  function renderClockPosition(color, bottomColor) {
    const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
      isBottom = color === bottomColor;
    (clockElement.classList.toggle('is-bottom', isBottom),
      clockElement.classList.toggle('is-top', !isBottom));
  }
  function renderClock(color, clockElement, timeElement) {
    const remainingMs = clockMs[color],
      isActive = activeClockColor === color && null === timedOutColor,
      isTimedOut = timedOutColor === color;
    (clockElement.classList.toggle('is-active', isActive),
      clockElement.classList.toggle('is-warning', remainingMs <= 3e4),
      clockElement.classList.toggle('is-danger', remainingMs <= 1e4),
      clockElement.classList.toggle('is-timeout', isTimedOut),
      (timeElement.textContent = (function formatClockTime(milliseconds) {
        const safeMilliseconds = Math.max(0, milliseconds);
        if (safeMilliseconds < 1e4)
```

```
        return `0:${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
      const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
      return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
    })(remainingMs)));
  }
  function renderStatus() {
    const historyLength = game.history().length;
    ((statusElement.textContent = (function getStatusText() {
      return game.isCheckmate()
        ? `${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
        : game.isStalemate()
          ? '스테일메이트 무승부'
          : game.isInsufficientMaterial()
            ? '기물 부족 무승부'
            : game.isThreefoldRepetition()
              ? '3회 반복 무승부'
              : game.isDrawByFiftyMoves()
                ? '50수 규칙 무승부'
                : game.isDraw()
                  ? '무승부'
                  : timedOutColor
                    ? [
                        `${colorName(timedOutColor)} 시간 초과`,
                        `${colorName(oppositeColor(timedOutColor))} 승리`,
                      ].join(' ')
                    : aiThinking
                      ? 'AI 계산 중'
                      : game.isCheck()
                        ? `${colorName(game.turn())} 체크`
                        : `${colorName(game.turn())} 차례`;
    })()),
      (turnMetaElement.textContent = colorName(game.turn())),
      (modeMetaElement.textContent =
        'local' === mode
          ? '로컬 2인'
          : ['AI 상대', `인간 ${colorName(humanColor)}`, presetName(aiSettings.preset)].join(
              ' · ',
            )),
      (moveMetaElement.textContent = [`${game.moveNumber()}수`, `${historyLength} half-move`].join(
        ' · ',
      )));
  }
  function renderAiCandidateDebug() {
    if ('ai' !== mode)
      return (
        (aiCandidatePanelElement.hidden = !0),
        (aiCandidateMetaElement.textContent = ''),
        (aiInfoStatusElement.textContent = ''),
        (aiInfoPresetElement.textContent = ''),
        (aiInfoHumanElement.textContent = ''),
        (aiInfoDepthElement.textContent = ''),
        (aiInfoNodesElement.textContent = ''),
        (aiInfoScoreElement.textContent = ''),
        void aiCandidateListElement.replaceChildren()
      );
    const debug = aiProgress?.debug;
    if (
      ((aiCandidatePanelElement.hidden = !1),
      (aiCandidateMetaElement.textContent = aiProgress
        ? `경과 ${formatSeconds(aiProgress.elapsedMs)}`
        : ''),
      (aiInfoStatusElement.textContent = (function getAiInfoStatusText() {
        return game.isGameOver() || null !== timedOutColor
          ? '대국 종료'
          : aiThinking
            ? aiProgress
              ? '계산 중'
              : '계산 준비 중'
            : game.turn() === humanColor
              ? '대기 중'
              : '응수 대기';
```

```
	})()),
	(aiInfoPresetElement.textContent = presetName(aiSettings.preset)),
	(aiInfoHumanElement.textContent = colorName(humanColor)),
	(aiInfoDepthElement.textContent = aiProgress ? String(aiProgress.depthReached) : '-'),
	(aiInfoNodesElement.textContent = aiProgress
		? `${(function formatNodes(nodes) {
			return nodes >= 1e6
				? `${(nodes / 1e6).toFixed(1)}M`
				: nodes >= 1e3
					? `${Math.round(nodes / 1e3)}k`
					: String(nodes);
		})(aiProgress.nodes)} nodes`
		: '-'),
	(aiInfoScoreElement.textContent = aiProgress ? formatDebugScore(aiProgress.score) : '-'),
	!debug || 0 === debug.candidates.length)
)
	return void aiCandidateListElement.replaceChildren();
aiCandidateMetaElement.textContent = [
	aiCandidateMetaElement.textContent,
	`best ${formatDebugScore(debug.bestScore)}`,
	`창 ${Math.round(debug.windowCp)}`,
	`p ${debug.ticketPower.toFixed(2)}`,
].join('  ');
const fragment = document.createDocumentFragment();
for (const candidate of debug.candidates) fragment.appendChild(createAiCandidateRow(candidate));
aiCandidateListElement.replaceChildren(fragment);
}
function createAiCandidateRow(candidate) {
const row = document.createElement('div');
((row.className = 'ai-candidate-row'),
	row.classList.toggle('is-selected', candidate.selected),
	row.classList.toggle('is-principal', candidate.principal),
	row.setAttribute('role', 'row'));
const sanCell = createAiCandidateCell('ai-candidate-san', candidate.san);
return (
	candidate.selected && sanCell.appendChild(createAiCandidateBadge('SEL')),
	candidate.principal && sanCell.appendChild(createAiCandidateBadge('PV')),
	row.append(
		sanCell,
		createAiCandidateCell('ai-candidate-score', formatDebugScore(candidate.score)),
		createAiCandidateCell(
			'ai-candidate-weight',
			(function formatDebugWeight(weight) {
				return 0 === weight
					? '0'
					: weight < 0.001
						? '<0.001'
						: weight.toFixed(3).replace(/0+$/, '').replace(/\.$/, '');
			})(candidate.tickets),
		),
		createAiCandidateCell(
			'ai-candidate-probability',
			(function formatDebugProbability(probability) {
				const percent = 100 * probability;
				return 0 === percent ? '0%' : percent < 0.1 ? '<0.1%' : `${percent.toFixed(1)}%`;
			})(candidate.probability),
		),
	),
	row
);
}
function createAiCandidateCell(className, text) {
const cell = document.createElement('span');
return (
	(cell.className = className),
	cell.setAttribute('role', 'cell'),
	(cell.textContent = text),
	cell
);
}
function createAiCandidateBadge(text) {
```

```
	const badge = document.createElement('span');
	return ((badge.className = 'ai-candidate-badge'), (badge.textContent = text), badge);
}
function getSquareClasses(square, lastMove) {
	const classes = ['board-square', 'light' === game.squareColor(square) ? 'is-light' : 'is-dark'],
		destinationMove = legalMoves.find((move) => move.to === square);
	return (
		selectedSquare === square && classes.push('is-selected'),
		destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
		!lastMove ||
			(lastMove.from !== square && lastMove.to !== square) ||
			classes.push('is-last-move'),
		(function isCheckedKing(square) {
			const piece = game.get(square);
			return Boolean(
				piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
			);
		})(square) && classes.push('is-check'),
		classes.join(' ')
	);
}
function getBoardOrientation() {
	return 'ai' === mode ? humanColor : game.turn();
}
function canUseBoard() {
	return !(
		game.isGameOver() ||
		null !== timedOutColor ||
		aiThinking ||
		pendingPromotion ||
		('local' !== mode && game.turn() !== humanColor)
	);
}
(boardElement.addEventListener('click', (event) => {
	const button = (function getSquareButton(target, boardElement) {
		if (!(target instanceof Element)) return null;
		const button = target.closest('.board-square');
		return button && boardElement.contains(button) ? button : null;
	})(event.target, boardElement);
	button &&
		(function isSquare(value) {
			return 'string' == typeof value && SQUARE_SET.has(value);
		})(button.dataset.square) &&
		(function handleSquareClick(square) {
			if (!canUseBoard()) return;
			const piece = game.get(square);
			if (selectedSquare) {
				const movesToDestination = legalMoves.filter((move) => move.to === square);
				return movesToDestination.length > 0
					? movesToDestination.some((move) => move.isPromotion())
						? ((pendingPromotion = {
								from: selectedSquare,
								to: square,
								moves: movesToDestination,
							}),
							void render())
						: void playMove({ from: selectedSquare, to: square })
					: piece?.color === game.turn()
						? void selectSquare(square)
						: (clearSelection(), void render());
			}
			piece?.color === game.turn() && selectSquare(square);
		})(button.dataset.square);
}),
	modeButtons.forEach((button) => {
		button.addEventListener('click', () => {
			const nextMode = button.dataset.mode;
			(function isGameMode(value) {
				return 'local' === value || 'ai' === value;
			})(nextMode) &&
				nextMode !== mode &&
```

```
    ((mode = nextMode), resetGame());
  });
});
colorButtons.forEach((button) => {
  button.addEventListener('click', () => {
    const nextColor = button.dataset.color;
    (function isColor(value) {
      return 'w' === value || 'b' === value;
    })(nextColor) &&
      nextColor !== humanColor &&
      ((humanColor = nextColor), resetGame());
  });
});
presetButtons.forEach((button) => {
  button.addEventListener('click', () => {
    const preset = button.dataset.preset;
    (function isAiPreset(value) {
      return 'easy' === value || 'normal' === value || 'hard' === value;
    })(preset) && updateAiSettings({ ...AI_PRESETS[preset] });
  });
});
clockPresetButtons.forEach((button) => {
  button.addEventListener('click', () => {
    const preset = button.dataset.clockPreset;
    (function isClockPreset(value) {
      return (
        '1/1' === value ||
        '3/2' === value ||
        '5/3' === value ||
        '10/5' === value ||
        '15/10' === value
      );
    })(preset) &&
      preset !== clockPreset &&
      ((clockPreset = preset), resetGame());
  });
});
aiDepthInput.addEventListener('input', () => {
  var value;
  updateAiSettings({
    ...aiSettings,
    maxDepth:
      ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value)))),
  });
});
aiTimeInput.addEventListener('input', () => {
  updateAiSettings({ ...aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
});
newGameButton.addEventListener('click', resetGame),
undoButton.addEventListener('click', function undoLastTurn() {
  (cancelAiMove(),
    (pendingPromotion = null),
    0 !== game.history().length
      ? ('ai' === mode && game.turn() === humanColor ? (game.undo(), game.undo()) : game.undo(),
        clearSelection(),
        resetClockState(),
        restartClockForCurrentTurn(),
        render())
      : render());
});
promotionChoices.addEventListener('click', (event) => {
  const target = event.target;
  if (!(target instanceof Element)) return;
  const button = target.closest('[data-promotion]'),
    promotion = button?.dataset.promotion;
  (function isPromotionPiece(value) {
    return 'q' === value || 'r' === value || 'b' === value || 'n' === value;
  })(promotion) &&
    (function finishPromotion(promotion) {
      if (!pendingPromotion) return;
      if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;
```

```
            const { from, to } = pendingPromotion;
            ((pendingPromotion = null), playMove({ from: from, to: to, promotion: promotion }));
        })(promotion);
    }),
    resetClockState(),
    restartClockForCurrentTurn(),
    render());
})();

```

```
==== index.html (6983 B / a6648d94f2cc) ====
<!doctype html>
<html lang="ko">
    <head>
        <meta charset="UTF-8" />
        <meta name="viewport" content="width=device-width, initial-scale=1.0" />
        <title>TS Chess</title>
        <script type="module" crossorigin src="/assets/index-CkN8R2zc.js"></script>
        <link rel="stylesheet" crossorigin href="/assets/index-av5gkzHq.css" />
    </head>
    <body>
        <main id="chess-app" class="chess-app">
            <aside class="control-panel" aria-label="게임 컨트롤">
                <section class="control-group" aria-label="모드">
                    <span class="control-label">모드</span>
                    <div class="segmented-control" role="group" aria-label="플레이 모드">
                        <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>
                        <button class="segmented-button" type="button" data-mode="ai">AI 상대</button>
                    </div>
                </section>

                <section id="color-controls" class="control-group" aria-label="진영">
                    <span class="control-label">진영</span>
                    <div class="segmented-control" role="group" aria-label="인간 진영">
                        <button class="segmented-button" type="button" data-color="w">백</button>
                        <button class="segmented-button" type="button" data-color="b">흑</button>
                    </div>
                </section>

                <section id="ai-controls" class="control-group ai-settings" aria-label="AI 설정">
                    <span class="control-label">AI 설정</span>
                    <div
                        class="segmented-control segmented-control-three"
                        role="group"
                        aria-label="AI 프리셋"
                    >
                        <button class="segmented-button" type="button" data-preset="easy">쉬움</button>
                        <button class="segmented-button" type="button" data-preset="normal">보통</button>
                        <button class="segmented-button" type="button" data-preset="hard">어려움</button>
                    </div>
                    <label class="range-control">
                        <span>최대 깊이 <strong id="ai-depth-value"></strong></span>
                        <input id="ai-depth-input" type="range" min="1" max="5" step="1" />
                    </label>
                    <label class="range-control">
                        <span>시간 제한 <strong id="ai-time-value"></strong></span>
                        <input id="ai-time-input" type="range" min="500" max="5000" step="250" />
                    </label>
                </section>

                <section class="control-group clock-settings" aria-label="시간 설정">
                    <span class="control-label">시간</span>
                    <div class="clock-preset-control" role="group" aria-label="시간 프리셋">
                        <button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>
                        <button class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>
                        <button class="segmented-button" type="button" data-clock-preset="5/3">5/3</button>
                        <button class="segmented-button" type="button" data-clock-preset="10/5">10/5</button>
                        <button class="segmented-button" type="button" data-clock-preset="15/10">15/10</button>
                    </div>
                </section>

```

```
        <section class="action-row" aria-label="게임 동작">
          <button id="new-game-button" class="primary-button" type="button">새 게임</button>
          <button id="undo-button" class="ghost-button" type="button">되돌리기</button>
        </section>
      </aside>

      <section class="board-stage" aria-label="체스판">
        <div id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>
      </section>

      <aside class="clock-panel" aria-label="체스 타이머">
        <div id="clock-display" class="clock-display">
          <div id="white-clock" class="clock-card">
            <span class="clock-side">백</span>
            <strong id="white-clock-time" class="clock-time"></strong>
          </div>
          <div id="black-clock" class="clock-card">
            <span class="clock-side">흑</span>
            <strong id="black-clock-time" class="clock-time"></strong>
          </div>
        </div>
      </aside>

      <aside class="info-panel" aria-label="게임 정보">
        <section class="status-panel" aria-live="polite">
          <span class="control-label">상태</span>
          <strong id="game-status" class="game-status"></strong>
          <dl class="status-grid">
            <div>
              <dt>차례</dt>
              <dd id="turn-meta"></dd>
            </div>
            <div>
              <dt>모드</dt>
              <dd id="mode-meta"></dd>
            </div>
            <div>
              <dt>수순</dt>
              <dd id="move-meta"></dd>
            </div>
          </dl>
        </section>

        <section id="ai-candidate-panel" class="ai-candidate-panel" aria-label="AI 정보" hidden>
          <div class="ai-candidate-header">
            <span class="control-label">AI 정보</span>
            <span id="ai-candidate-meta" class="ai-candidate-meta"></span>
          </div>
          <dl class="ai-info-grid">
            <div>
              <dt>상태</dt>
              <dd id="ai-info-status"></dd>
            </div>
            <div>
              <dt>난이도</dt>
              <dd id="ai-info-preset"></dd>
            </div>
            <div>
              <dt>인간</dt>
              <dd id="ai-info-human"></dd>
            </div>
            <div>
              <dt>depth</dt>
              <dd id="ai-info-depth"></dd>
            </div>
            <div>
              <dt>nodes</dt>
              <dd id="ai-info-nodes"></dd>
            </div>
            <div>
              <dt>score</dt>
```

```
        <dd id="ai-info-score"></dd>
      </div>
    </dl>
    <div class="ai-candidate-table" role="table" aria-label="AI 후보 수 평가">
      <div class="ai-candidate-row ai-candidate-head" role="row">
        <span role="columnheader">SAN</span>
        <span role="columnheader">cp</span>
        <span role="columnheader">weight</span>
        <span role="columnheader">prob</span>
      </div>
      <div id="ai-candidate-list" class="ai-candidate-list"></div>
    </div>
  </section>

  <section class="history-panel" aria-label="기보">
    <div class="history-header">
      <span class="control-label">기보</span>
    </div>
    <div id="move-list" class="move-list"></div>
  </section>
</aside>

<div id="promotion-overlay" class="promotion-overlay" hidden>
  <div
    class="promotion-dialog"
    role="dialog"
    aria-modal="true"
    aria-labelledby="promotion-title"
  >
    <h2 id="promotion-title">승격</h2>
    <div
      id="promotion-choices"
      class="promotion-choices"
      role="group"
      aria-label="승격 기물"
    ></div>
  </div>
</div>
</main>
</body>
</html>
```